

MBS-fast

Руководство пользователя и разработчика

Геометрическая и физическая оптика для многогранных частиц

Документ описывает физическую модель, численные алгоритмы, сборку, параметры запуска, форматы данных, ускорение на CPU и GPU, а также обязательные проверки точности. В командах используются канонические имена параметров из текущего интерфейса программы; устаревшие сокращения указаны отдельно и нужны только для воспроизводимости старых сценариев.

Редакция от 12 августа 2026 г.

Документ для актуального интерфейса командной строки

О руководстве

MBS-fast предназначен для расчёта дальнего рассеяния света многогранными частицами. Геометрическая стадия строит и трассирует полигональные пучки, перенося комплексную матрицу Джонса через отражения и преломления. В режиме физической оптики (РО) каждый выходной пучок дополняется дифракционным интегралом по его апертуре. После когерентного суммирования амплитуд результат переводится в матрицу Мюллера и, при необходимости, усредняется по ориентациям частицы.

Руководство рассчитано на два способа чтения. Для первого запуска достаточно разделов «Быстрый старт», «Сборка и исполняемые файлы», «Параметры командной строки» и «Обязательные проверки результата». Разделы о трассировке, коэффициентах Френеля, дифракционном интеграле и реализации на CPU/GPU предназначены для проверки физики и модификации кода. Приложения содержат формулы метрик и контрольный список для воспроизводимого расчёта.

Содержание

О руководстве	i
I Практическая работа	1
1 Быстрый старт	1
II Физическая модель и геометрия	2
2 Назначение программы	2
3 Координатные системы	4
4 Трассировка лучей	5
5 Коэффициенты Френеля	7
6 Матрицы Джонса и диады	8
7 Переход к матрице Мюллера	10
8 Дифракция в приближении физической оптики	12
9 Обрезка пучков в невыпуклых частицах и агрегатах	14
10 Размер частицы и масштабирование геометрии	15

III	Ориентации и угловые сетки	19
11	Ориентационное усреднение	19
12	Угловая сетка рассеяния	25
IV	Реализация и производительность	26
13	Реализация на CPU и AVX	26
14	Реализация на GPU	28
V	Сборка, запуск и данные	31
15	Сборка и исполняемые файлы	31
16	Запуск и первичная диагностика	32
17	Параметры командной строки	33
18	Выходные файлы и интегральные величины	36
VI	Точность, проверка и воспроизводимость	38
19	Обязательные проверки результата	38
20	Оценка погрешности и выбор параметров	38
21	Ускорения и контролируемые приближения	41
22	Диагностика и воспроизводимость	45
VII	Приложения	49
A	Связь с файлами кода	49
B	Контрольные формулы для отчётов	50
C	Контрольный список перед публикацией результатов	51

Часть I

Практическая работа

1. Быстрый старт

1.1. Сборка проверочного бинарного файла

Для воспроизводимой проверки сначала собирают CPU-версию и запускают матрицу коротких тестов интерфейса:

```
make -C cpu -j
make test_release
cpu/bin/mbs_po_mpi --version
```

GPU-версия двойной точности собирается отдельной целью. Именно её следует использовать для проверки точек 0° и 180° и для сравнения с CPU:

```
make gpu_double_fast -j
gpu/bin/mbs_po_gpu_double_fast --version
```

1.2. Первый расчёт

Следующая команда считает гексагональный столбик длиной 100 мкм и диаметром 70 мкм при $\lambda = 0.532$ мкм. Сетка ориентаций и углов рассеяния строится из дифракционной оценки; широтная сетка уменьшает избыточное число азимутальных узлов около полюсов.

```
CUDA_VISIBLE_DEVICES=0 \
gpu/bin/mbs_po_gpu_double_fast \
  --method po \
  --particle 1 100 70 \
  --refractive-index 1.3116 0 \
  --wavelength-um 0.532 \
  --max-reflections 8 \
  --diffraction-sampling 2 \
  --latitude-phi-grid \
  --backend cuda --threads 16 \
  --cutoff-profile off \
  --output results/column_L100 --close
```

Это контрольная, а не самая быстрая конфигурация. Профили отсечения, FFT и многопроцессорное распределение включают только после сравнения с таким прямым расчётом на нескольких характерных размерах.

1.3. Что проверить после завершения

Расчёт можно использовать дальше, только если одновременно выполнены следующие условия:

1. процесс завершился с нулевым кодом, а выходной файл содержит все строки по θ от 0° до 180° ;
2. в логе нет сообщения о достижении ограничения числа пучков или верхней границы адаптивного поиска;
3. отчёт интегральных величин не содержит статуса `check_csca_integration`;
4. более мелкая сетка по ориентациям и углам меняет требуемые элементы матрицы Мюллера меньше заданной погрешности;
5. для новой формы хотя бы одна небольшая задача совпадает между CPU и GPU двойной точности.

Завершившийся процесс ещё не означает сошедшийся физический результат. Глубина трассировки, угловая сетка, ориентационное усреднение, отсечения слабых пучков и численная точность проверяются независимо.

Часть II

Физическая модель и геометрия

2. Назначение программы

MBS-fast решает задачу дальнего рассеяния на частице с плоскими гранями. В отличие от метода дискретных диполей программа не заполняет объём частицы расчётными узлами. Она строит геометрические пучки, возникающие после отражений и преломлений на гранях. Каждый пучок хранит полигон апертуры, направление распространения, оптический путь и комплексную матрицу Джонса. После трассировки для каждого выходного пучка вычисляется дифракционный интеграл физической оптики. Амплитуды суммируются когерентно и преобразуются в матрицу Мюллера.

2.1. Физические уровни модели

В расчете используются четыре уровня приближения.

Таблица 1: Физические уровни модели MBS-fast.

Уровень	Что считается	Где появляется в коде
Геометрическая оптика	Пучки отражаются и преломляются на плоских гранях по законам Снеллиуса; малозначимые ветви могут быть отсечены заданным критерием.	<code>src/scattering</code> , <code>Splitting.cpp</code> .

Уровень	Что считается	Где появляется в коде
Френель и Джонс	Поляризационная амплитуда каждой ветви хранится как комплексная матрица Джонса.	Beam::J, Splitting.cpp.
Физическая оптика	Выходной пучок создаёт дальнее поле, вычисляемое интегралом по его полигональной апертуре.	HandlerPO::ApplyDiffraction*, ядра CUDA.
Ориентационное усреднение	Для ансамбля случайно ориентированных частиц вычисляется взвешенная сумма матриц Мюллера по квадратной или квазислучайной сетке.	HandlerPOTotal, TracerPOTotal.

В режиме РО пучки одной ориентации по умолчанию суммируются когерентно на уровне амплитуд Джонса. Только после этой суммы строится матрица Мюллера. Параметр `--incoherent` меняет физическую постановку: матрицы Мюллера отдельных пучков складываются без интерференционных членов.

2.2. Обозначения

Таблица 2: Базовые обозначения, используемые дальше.

Символ	Смысл
λ	Длина волны в микрометрах, задаётся параметром <code>--wavelength-um</code> .
$k = 2\pi/\lambda$	Волновое число во внешней среде.
$m = n + i\kappa$	Комплексный показатель преломления частицы относительно внешней среды, задаётся параметром <code>--refractive-index Re Im</code> .
d	Направление распространения текущего луча или пучка.
n	Внешняя нормаль грани. В коде знак выбирается так, чтобы ветвление было устойчивым для текущей стороны грани.
θ, ϕ	Углы направления наблюдения в дальней зоне.
$\mathbf{s}(\theta, \phi)$	Единичный вектор направления наблюдения.
J	Комплексная матрица Джонса 2×2 .
M	Вещественная матрица Мюллера 4×4 .
A	Площадь или полигон апертуры; смысл уточняется в конкретной формуле.
N_b	Число выходных пучков для одной ориентации.
N_{ori}	Число ориентаций в усреднении.



CPU подготавливает лучи и пучки; CPU или GPU вычисляет дифракцию на сетке направлений

Рис. 1: Последовательность расчёта: геометрия частицы, трассировка, коэффициенты Френеля и матрицы Джонса, дифракция, когерентная сумма, матрица Мюллера и усреднение по ориентациям.

3. Координатные системы

3.1. Лабораторная система

Лабораторная система фиксирована относительно падающего света. Обычно падающий луч направлен вдоль одной из осей, а частица поворачивается матрицей Эйлера. Для любого вектора $\mathbf{x}$ в лабораторной системе используются скалярное произведение $\mathbf{a} \cdot \mathbf{b}$ и векторное произведение $\mathbf{a} \times \mathbf{b}$. Все направления лучей нормируются:

$$\|\mathbf{d}\| = 1, \quad \|\mathbf{s}\| = 1, \quad \|\mathbf{n}\| = 1.$$

3.2. Система частицы и повороты Эйлера

Пусть $\mathbf{x}_p$ — координаты вершины в системе частицы, а $\mathbf{x}_l$ — координаты в лабораторной системе. Поворот задается матрицей

$$\mathbf{x}_l = \mathbf{R}(\alpha, \beta, \gamma) \mathbf{x}_p.$$

Полная ориентация твёрдого тела задаётся тремя углами Эйлера или единичным кватернионом. В основной двухугловой квадратуре MBS-fast явно перебираются β и γ , а усреднение по лабораторному углу α представлено азимутальной сеткой направления рассеяния. Поэтому уменьшение числа азимутальных точек меняет не только вывод кривой, но и точность этого эквивалентного усреднения. Режим `--so3-quaternion` непосредственно строит выборку в полной группе вращений $SO(3)$.

Удобная запись через элементарные повороты:

$$\mathbf{R}(\alpha, \beta, \gamma) = \mathbf{R}_z(\alpha) \mathbf{R}_y(\beta) \mathbf{R}_z(\gamma).$$

Здесь

$$\mathbf{R}_z(\chi) = \begin{pmatrix} \cos \chi & -\sin \chi & 0 \\ \sin \chi & \cos \chi & 0 \\ 0 & 0 & 1 \end{pmatrix}, \quad \mathbf{R}_y(\chi) = \begin{pmatrix} \cos \chi & 0 & \sin \chi \\ 0 & 1 & 0 \\ -\sin \chi & 0 & \cos \chi \end{pmatrix}.$$

Таблица 3: Смысл углов Эйлера.

Символ	Смысл
α	Поворот вокруг лабораторной оси падающего луча или начальный азимут, если используется полная $SO(3)$ -сетка.
β	Наклон оси частицы относительно падающего луча; мера содержит якобиан $\sin \beta$.
γ	Поворот частицы вокруг собственной оси после наклона. При $\beta = 0$ и $\beta = \pi$ все значения γ геометрически эквивалентны.

3.3. Система грани и апертуры

Для каждого выходного пучка строится локальная система апертуры. Нормаль апертуры $\mathbf{n}_b$ берется из последней грани или из нормали пучка, а две оси в плоскости обозначим $\mathbf{h}_b$ и $\mathbf{v}_b$:

$$\mathbf{h}_b \cdot \mathbf{n}_b = 0, \quad \mathbf{v}_b \cdot \mathbf{n}_b = 0, \quad \mathbf{h}_b \cdot \mathbf{v}_b = 0.$$

Точка $\mathbf{r}$ в лабораторной системе проектируется в координаты апертуры:

$$x = (\mathbf{r} - \mathbf{r}_c) \cdot \mathbf{h}_b, \quad y = (\mathbf{r} - \mathbf{r}_c) \cdot \mathbf{v}_b,$$

где $\mathbf{r}_c$ — центр полигона пучка. Именно эти x, y попадают в `BeamEdgeData`: массивы вершин, наклоны ребер и признаки валидности ребер.

3.4. Система рассеяния

Для направления наблюдения $\mathbf{s}$ строятся две поперечные оси. В коде используется опорный вектор $\mathbf{v}_f$ и второй вектор

$$\mathbf{v}_t = \frac{\mathbf{v}_f \times \mathbf{s}}{\|\mathbf{v}_f \times \mathbf{s}\|}.$$

Если направление близко к полюсу и знаменатель мал, используется предельная ветвь: при вырожденном азимуте базис рассеяния не определяется однозначно.

4. Трассировка лучей

4.1. Пересечение луча с гранью

Луч задается начальной точкой $\mathbf{r}_0$ и направлением $\mathbf{d}$:

$$\mathbf{r}(t) = \mathbf{r}_0 + t\mathbf{d}, \quad t \geq 0.$$

Грань лежит в плоскости

$$(\mathbf{r} - \mathbf{p}_f) \cdot \mathbf{n}_f = 0,$$

где $\mathbf{p}_f$ — любая точка грани, $\mathbf{n}_f$ — нормаль грани. Формула пересечения:

$$t_f = \frac{(\mathbf{p}_f - \mathbf{r}_0) \cdot \mathbf{n}_f}{\mathbf{d} \cdot \mathbf{n}_f}.$$

Кандидат допустим, если $t_f > 0$, знаменатель не близок к нулю, а точка $\mathbf{r}(t_f)$ лежит внутри полигона грани. Для невыпуклой частицы этого недостаточно: найденная область может быть частично или полностью закрыта другой гранью.

4.2. Пучок как полигон

MBS-fast трассирует не одиночный луч, а полигональный пучок. Его апертура пересекается с полигоном грани. Если P_{in} — входной полигон, а F — грань в той же плоскости, то область попадания равна

$$P_{\text{hit}} = P_{\text{in}} \cap F.$$

Площадь $|P_{\text{hit}}|$ входит в критерии отсечения и нормировки. Ветвь с вырожденным полигоном удаляется.

4.3. Отражение и преломление

Пусть $\cos A = \mathbf{n} \cdot \mathbf{d}$. В `Splitting.cpp` используется вектор

$$\mathbf{r} = \frac{\mathbf{d}}{\cos A} - \mathbf{n},$$

после чего направление отраженной ветви записано как

$$\mathbf{d}_{\text{refl}} = \frac{\mathbf{r} - \mathbf{n}}{\|\mathbf{r} - \mathbf{n}\|}.$$

Такая запись эквивалентна обычной формуле зеркального отражения после учета знаковой конвенции нормали в коде. Для преломленной ветви код строит

$$s = \frac{1}{m_{\text{eff}}^2 \cos^2 A} - \|\mathbf{r}\|^2, \quad \mathbf{d}_{\text{refr}} = \frac{\mathbf{r}/\sqrt{s} + \mathbf{n}}{\|\mathbf{r}/\sqrt{s} + \mathbf{n}\|}.$$

Если s становится неположительным, обычная преломленная ветвь отсутствует, и включается ветка полного внутреннего отражения.

4.4. Оптический путь

Фаза пучка включает оптический путь

$$\Phi_{\text{path}} = k L_{\text{opt}}.$$

Для внешней среды вклад длины равен геометрической длине. Для внутреннего участка длина умножается на эффективную вещественную часть показателя:

$$L_{\text{opt},\text{in}} \approx \sqrt{\text{Re}(m^2)} L_{\text{geom}}.$$

В поглощающем варианте дополнительно используется мнимая часть показателя и интеграл с затуханием вдоль внутреннего пути.

4.5. Ограничение глубины

Параметр `--max-reflections N` ограничивает глубину внутренних отражений и преломлений. Если g — поколение дерева пучков, ветви с $g > N$ не трассируются. Без геометрического отсечения число ветвей могло бы расти как

$$N_{\text{beam}} \sim 2^g$$

но на практике рост ограничивают геометрия, поглощение и заданные пороги.

5. Коэффициенты Френеля

5.1. Вертикальная и горизонтальная поляризация

При каждом взаимодействии с гранью матрица Джонса умножается на два комплексных коэффициента: для локальной вертикальной и горизонтальной поляризаций. В коде они обозначены `cv/ch` или `Tv/Th`. Это диагональный оператор в локальном базисе Джонса:

$$\mathbf{J}_{\text{new}} = \begin{pmatrix} c_v & 0 \\ 0 & c_h \end{pmatrix} \mathbf{J}_{\text{old}} \quad \text{или} \quad \mathbf{J}_{\text{old}} \begin{pmatrix} c_v & 0 \\ 0 & c_h \end{pmatrix},$$

в зависимости от внутренней конвенции хранения строк и столбцов.

5.2. Обычное отражение и прохождение

В `ComputeRegularJonesParams` после вычисления $\cos G = \mathbf{n} \cdot \mathbf{d}_{\text{out}}$ используются величины

$$a_0 = m \cos A, \quad a_1 = m \cos G,$$

$$D_v = a_1 + \cos A, \quad D_h = a_0 + \cos G.$$

Для проходящей ветви:

$$c_v^{(T)} = \frac{2a_0}{D_v}, \quad c_h^{(T)} = \frac{2a_0}{D_h}.$$

Для отраженной внутренней ветви:

$$c_v^{(R)} = \frac{\cos A - a_1}{D_v}, \quad c_h^{(R)} = \frac{a_0 - \cos G}{D_h}.$$

Таблица 4: Обозначения в формулах обычной ветви Френеля.

Символ	Смысл
m	Комплексный показатель преломления частицы.
$\cos A$	Косинус угла между нормалью и падающим направлением в конвенции кода.
$\cos G$	Косинус угла между нормалью и выходящим направлением.

Символ	Смысл
D_v, D_h	Знаменатели Френеля для двух локальных поляризаций.
$c_v^{(T)}, c_h^{(T)}$	Множители Джонса для прошедшей ветви.
$c_v^{(R)}, c_h^{(R)}$	Множители Джонса для отражённой ветви.

5.3. Нормальное падение

При нормальном падении код использует упрощенные коэффициенты:

$$c_T = \frac{2m}{1+m}, \quad c_R = \frac{1-m}{1+m}.$$

Для одной из поляризаций отраженная ветвь получает дополнительный знак, что связано с ориентацией локального поляризационного базиса после отражения:

$$\mathbf{J}_{\text{refl}} \leftarrow \begin{pmatrix} c_R & 0 \\ 0 & -c_R \end{pmatrix} \mathbf{J}.$$

5.4. Полное внутреннее отражение

При полном внутреннем отражении вычисляется

$$b = m_{\text{eff}}^2(1 - \cos^2 A) - 1, \quad q = i\sqrt{\max(b, 0)}.$$

Затем

$$c_v = \frac{\cos A - mq}{mq + \cos A}, \quad c_h = \frac{m \cos A - q}{m \cos A + q}.$$

В непоглощающей среде их модуль близок к единице, но сохраняется фазовый сдвиг, определяющий поляризационные элементы матрицы Мюллера.

5.5. Старая конвенция знака Френеля

Параметр `--legacy-sign` включает прежнюю конвенцию знака в направлении вперёд. Она нужна только для воспроизведения старых результатов. В новых расчётах следует использовать поведение по умолчанию.

6. Матрицы Джонса и диады

6.1. Матрица пучка

Каждый пучок несет

$$\mathbf{J}_{\text{beam}} = \begin{pmatrix} J_{vv} & J_{vh} \\ J_{hv} & J_{hh} \end{pmatrix}.$$

Это амплитудная матрица: интенсивность определяется квадратами модулей, а интерференция сохраняется до суммирования всех матриц Джонса и последующего преобразования в матрицу Мюллера.

6.2. Фазовый множитель

Для внутреннего пучка функция `ComputeFnJones` использует фазу

$$\mathbf{F}_n(\mathbf{s}) = \mathbf{J}_{\text{beam}} \exp[ik(L_{\text{proj}} - \mathbf{s} \cdot \mathbf{r}_c)].$$

Здесь L_{proj} хранится в поле `info.projLenght`, а $\mathbf{r}_c$ — центр пучка. Для внешнего пучка используется накопленный оптический путь:

$$\mathbf{F}_n = \mathbf{J}_{\text{beam}} \exp(ikL_{\text{opt}}).$$

6.3. Поворот матрицы Джонса в базис рассеяния

В `HandlerP0::RotateJones` строятся векторы:

$$\mathbf{v}_t = \frac{\mathbf{v}_f \times \mathbf{s}}{\|\mathbf{v}_f \times \mathbf{s}\|},$$

$$\mathbf{N}_T = \mathbf{n} \times \mathbf{e}_T, \quad \mathbf{N}_P = \mathbf{n} \times \mathbf{e}_P,$$

$$\mathbf{D}_T = \mathbf{d}_b \times \mathbf{e}_T, \quad \mathbf{D}_P = \mathbf{d}_b \times \mathbf{e}_P.$$

Здесь $\mathbf{e}_T$ хранится в `info.beamBasis`, а $\mathbf{e}_P$ — в `beam.polarizationBasis`. Далее используется тождество тройного векторного произведения:

$$\mathbf{c}_T = \mathbf{s} \times \mathbf{N}_T + \mathbf{n} \times \mathbf{D}_T - \mathbf{s} [\mathbf{s} \cdot (\mathbf{n} \times \mathbf{D}_T)],$$

$$\mathbf{c}_P = \mathbf{s} \times \mathbf{N}_P + \mathbf{n} \times \mathbf{D}_P - \mathbf{s} [\mathbf{s} \cdot (\mathbf{n} \times \mathbf{D}_P)].$$

Матрица поворота:

$$\mathbf{R}_{\text{out}} = \frac{1}{2} \begin{pmatrix} \mathbf{c}_T \cdot \mathbf{v}_t & \mathbf{c}_P \cdot \mathbf{v}_t \\ \mathbf{c}_T \cdot \mathbf{v}_f & \mathbf{c}_P \cdot \mathbf{v}_f \end{pmatrix}.$$

6.4. Полный вклад пучка в матрицу Джонса

В `ApplyDiffractionFast` итоговый вклад одного пучка:

$$\mathbf{J}_b(\theta, \phi) = F_{\text{edge},b}(\theta, \phi) \mathbf{R}_{\text{out},b}(\theta, \phi) \mathbf{F}_{n,b}(\theta, \phi).$$

Таблица 5: Множители полного вклада Джонса.

Множитель	Смысл
$F_{\text{edge},b}$	Скалярный интеграл физической оптики по полигональной апертуре пучка b .
$\mathbf{R}_{\text{out},b}$	Переход из локального базиса Джонса пучка в базис рассеяния.

Множитель	Смысл
$\mathbf{F}_{n,b}$	Накопленная матрица Джонса с фазой оптического пути и поправкой на направление наблюдения.

6.5. Экспериментальная ветвь Карчевского

Параметр `--karczewski` заменяет `RotateJones` на `KarczewskiJones`. Эта ветвь экспериментальна. Она сохраняет M_{11} , поскольку не меняет норму Фробениуса оператора Джонса:

$$M_{11} = \frac{1}{2} \|\mathbf{J}\|_F^2 = \frac{1}{2} \sum_{i=1}^2 \sum_{j=1}^2 |J_{ij}|^2.$$

Поляризационные элементы M_{33} , M_{34} и M_{44} могут отличаться, поскольку меняется структура матрицы Джонса в базисе рассеяния.

7. Переход к матрице Мюллера

7.1. Полная формула из кода

Пусть

$$\mathbf{J} = \begin{pmatrix} a & b \\ c & d \end{pmatrix}.$$

В `src/math/Mueller.cpp` матрица Мюллера строится из билинейных комбинаций в следующей конвенции:

$$\begin{aligned} M_{00} &= \frac{|a|^2 + |b|^2 + |c|^2 + |d|^2}{2}, & M_{01} &= \frac{|a|^2 - |b|^2 + |c|^2 - |d|^2}{2}, \\ M_{10} &= \frac{|a|^2 + |b|^2 - |c|^2 - |d|^2}{2}, & M_{11} &= \frac{|a|^2 - |b|^2 - |c|^2 + |d|^2}{2}. \end{aligned}$$

Далее:

$$\begin{aligned} M_{02} &= -\operatorname{Re}(a\bar{b}) - \operatorname{Re}(d\bar{c}), & M_{03} &= \operatorname{Im}(d\bar{c}) - \operatorname{Im}(a\bar{b}), \\ M_{12} &= \operatorname{Re}(d\bar{c}) - \operatorname{Re}(a\bar{b}), & M_{13} &= -\operatorname{Im}(a\bar{b}) - \operatorname{Im}(d\bar{c}), \\ M_{20} &= -\operatorname{Re}(a\bar{c}) - \operatorname{Re}(d\bar{b}), & M_{21} &= \operatorname{Re}(d\bar{b}) - \operatorname{Re}(a\bar{c}), \\ M_{30} &= \operatorname{Im}(a\bar{c}) - \operatorname{Im}(d\bar{b}), & M_{31} &= \operatorname{Im}(d\bar{b}) + \operatorname{Im}(a\bar{c}), \\ M_{22} &= \operatorname{Re}(a\bar{d}) + \operatorname{Re}(b\bar{c}), & M_{23} &= \operatorname{Im}(a\bar{d}) - \operatorname{Im}(b\bar{c}), \\ M_{32} &= -\operatorname{Im}(a\bar{d}) - \operatorname{Im}(b\bar{c}), & M_{33} &= \operatorname{Re}(a\bar{d}) - \operatorname{Re}(b\bar{c}). \end{aligned}$$

Таблица 6: Обозначения при переходе от матрицы Джонса к матрице Мюллера.

Символ	Смысл
a, b, c, d	Комплексные элементы итоговой матрицы Джонса для одного направления наблюдения.
$\bar{z}$	Комплексное сопряжение.
$ z ^2 = z\bar{z}$	Квадрат модуля комплексной амплитуды.
$\text{Re}(z), \text{Im}(z)$	Вещественная и мнимая части.
M_{ij}	Элементы Мюллера в нулевой индексации кода; в файлах записываются как $M11, M12, \dots, M44$.

7.2. Когерентная и некогерентная сумма

По умолчанию для одной ориентации:

$$\mathbf{J}_{\text{tot}}(\theta, \phi) = \sum_{b=1}^{N_b} \mathbf{J}_b(\theta, \phi), \quad \mathbf{M}(\theta, \phi) = \mathcal{M}(\mathbf{J}_{\text{tot}}).$$

С параметром `--incoherent`:

$$\mathbf{M}_{\text{incoh}}(\theta, \phi) = \sum_{b=1}^{N_b} \mathcal{M}(\mathbf{J}_b(\theta, \phi)).$$

Разница принципиальная: в первом случае сохраняются интерференционные члены между разными пучками, во втором они исчезают.

7.3. Повороты матрицы Мюллера

Поворот базиса Стокса на угол χ действует только на компоненты Q, U :

$$\mathbf{R}_M(\chi) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \chi & -\sin \chi & 0 \\ 0 & \sin \chi & \cos \chi & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

`RightRotateMueller` умножает матрицу справа и меняет входной базис Стокса:

$$\mathbf{M}' = \mathbf{M} \mathbf{R}_M(\chi_{\text{in}}).$$

`LeftRotateMueller` умножает слева, то есть меняет выходной базис:

$$\mathbf{M}' = \mathbf{R}_M(\chi_{\text{out}}) \mathbf{M}.$$

`RotateMueller` делает обе операции в одной функции:

$$\mathbf{M}' = \mathbf{R}_M(\chi_{\text{out}}) \mathbf{M} \mathbf{R}_M(\chi_{\text{in}}).$$

8. Дифракция в приближении физической оптики

8.1. Интеграл по апертуре

Для выходного пучка с апертурой A_b скалярная часть РО-дальнего поля записывается как

$$F_b(\mathbf{q}) = \int_{A_b} \exp(i\mathbf{k} \cdot \mathbf{r}) dA, \quad \mathbf{q} = \mathbf{d}_b - \mathbf{s}.$$

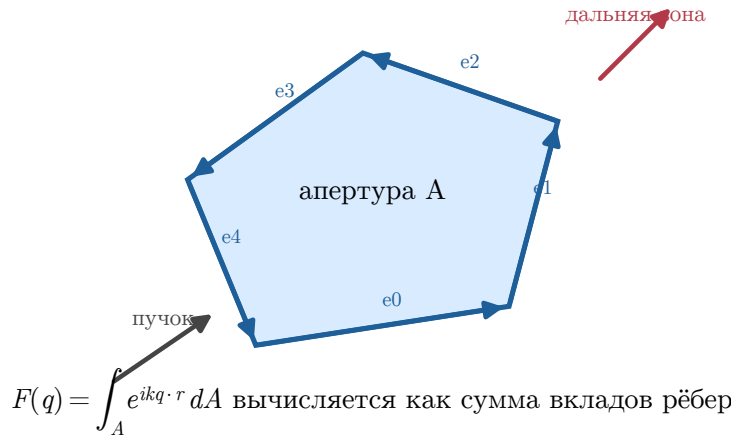


Рис. 2: Интеграл по апертуре сводится к сумме вкладов рёбер полигона. Такой способ быстрее и точнее прямого численного интегрирования по площади.

8.2. Переход к ребрам полигона

В системе апертуры:

$$A_q = \mathbf{q} \cdot \mathbf{h}_b, \quad B_q = \mathbf{q} \cdot \mathbf{v}_b.$$

Для вершины j :

$$\Phi_j = k(A_q x_j + B_q y_j).$$

Если ребро описано через $y = m_j x + c_j$, то один из вариантов суммы по рёбрам имеет вид

$$S_x = \frac{1}{B_q} \sum_j \frac{\exp(i\Phi_{j+1}) - \exp(i\Phi_j)}{A_q + m_j B_q}.$$

Если удобнее параметризовать ребро как $x = \mu_j y + \eta_j$, используется симметричная форма

$$S_y = -\frac{1}{A_q} \sum_j \frac{\exp(i\Phi_{j+1}) - \exp(i\Phi_j)}{A_q \mu_j + B_q}.$$

В коде выбор между ветками зависит от того, какой знаменатель устойчивее. Для малых A_q и B_q используется предельная формула площади:

$$F_b(\mathbf{0}) \approx C_k |A_b|,$$

где C_k — комплексный нормировочный множитель, соответствующий выбранной форме интеграла в коде.

Таблица 7: Обозначения в интеграле по рёбрам.

Символ	Смысл
$\mathbf{q}$	Разность направления пучка и направления наблюдения.
A_q, B_q	Компоненты $\mathbf{q}$ в плоскости апертуры.
(x_j, y_j)	Вершина полигона пучка в системе апертуры.
m_j, μ_j	Наклоны ребра: dy/dx и dx/dy .
Φ_j	Фаза комплексной экспоненты в вершине j .
S_x, S_y	Две эквивалентные формы суммы по рёбрам, выбираемые по устойчивости.

8.3. Почему важны точки 0 и 180 градусов

В направлениях точно вперёд и точно назад часть базисов и знаменателей становится вырожденной или почти вырожденной:

$$\|\mathbf{v}_f \times \mathbf{s}\| \rightarrow 0, \quad A_q \rightarrow 0, \quad B_q \rightarrow 0.$$

Поэтому в этих точках особенно важны:

- одинаковая обработка концов углового диапазона на CPU и GPU;
- одинаковая формула предела интеграла по рёбрам;
- одинаковый вес γ на полюсах по β при `--pole`;
- отсутствие расхождения в знаке коэффициентов Френеля и в соглашении о фазе;
- одинаковая точность: одинарная точность может заметно отличаться от двойной именно в почти вырожденных ветвях.

8.4. Поглощение

Если $\text{Im}(m) > 0$ или указан `--abs`, внутренняя часть пучка получает затухание:

$$\exp(ikmL) = \exp(ik \text{Re}(m)L) \exp(-k \text{Im}(m)L).$$

Тогда вклад длинных внутренних путей уменьшается. В численном смысле это помогает сходимости дерева лучей, но меняет физическую интенсивность и поляризационные элементы.

9. Обрезка пучков в невыпуклых частицах и агрегатах

9.1. Затенение апертуры другими гранями

Для выпуклой частицы пересечение пучка с ближайшей гранью обычно однозначно. У невыпуклой частицы другая грань может закрывать часть апертуры. Тогда из полигона текущего пучка вычитают проекцию блокирующей грани:

$$P_{\text{vis}} = P_{\text{subj}} \setminus \bigcup_{m=1}^{N_c} \Pi_{\text{subj}}(C_m).$$

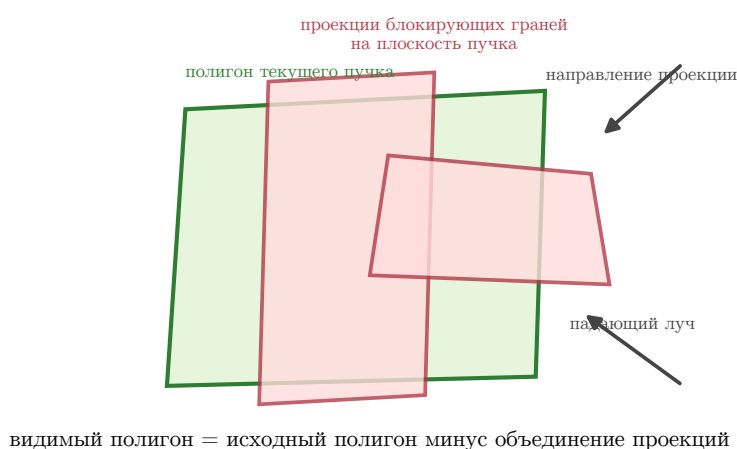


Рис. 3: Обрезка невыпуклого пучка: проекция блокирующей грани удаляет закрытую часть его полигона.

Таблица 8: Объекты геометрической обрезки.

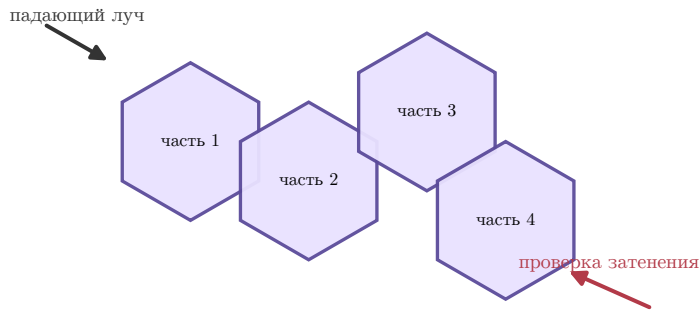
Символ	Смысл
P_{subj}	Полигон текущего пучка, который проверяется на видимость.
C_m	Блокирующая грань или полигон, потенциально закрывающие текущий пучок.
Π_{subj}	Проекция блокирующего объекта на плоскость текущего пучка вдоль направления распространения.
P_{vis}	Видимая часть полигона после вычитания проекций.
N_c	Число кандидатов после быстрых фильтров.

9.2. Агрегаты

В агрегате блокирующая грань может принадлежать другой компоненте:

$$C_m \in \bigcup_{p=1}^{N_{\text{part}}} \mathcal{F}_p,$$

где $\mathcal{F}_p$ — множество граней компоненты p .



в агрегате грань может быть закрыта гранями другой части

Рис. 4: Для агрегата затенение идет не только внутри одной компоненты, но и между разными компонентами.

9.3. Предварительный отбор по проекциям AABV

Точное вычитание полигонов требует значительных затрат. Поэтому `--trace-prefilter` сначала строит ограничивающий прямоугольник проекции блокирующей грани:

$$B(C_m) = [x_{\min}, x_{\max}] \times [y_{\min}, y_{\max}].$$

Если

$$B(C_m) \cap B(P_{\text{subj}}) = \emptyset,$$

то точная обрезка не нужна. Это не меняет физику, а только уменьшает число дорогих геометрических операций.

10. Размер частицы и масштабирование геометрии

В этом разделе различаются геометрические размеры, задаваемые генератору, и производные характеристики, используемые для сопоставления частиц. Масштаб всегда применяется к фактически построенной или прочитанной геометрии.

10.1. Встроенные частицы и геометрия из файла

Частица задаётся либо параметром `--particle`, после которого следуют тип и его параметры, либо параметром `--particle-file` с именем файла. В первом случае геометрию создаёт встроенный генератор, во втором она читается из файла. Одновременное задание двух источников запрещено. Параметр `--save-geometry FILE` сохраняет фактически использованную геометрию после масштабирования и определения её выпуклости; затем расчёт продолжается.

Таблица 9: Основные встроенные частицы.

Тип	Форма	Параметры
1	Гексагональный столбик или пластинка	L D : осевая длина и диаметр между противоположными вершинами. Отношение L/D определяет вытянутую или сплюснутую форму.
2	Пуля	L D : полная осевая длина и диаметр; высота пирамидальной головки вычисляется из фиксированного угла 62° .
3	Розетка из пуль	L D [CAP]: шесть одинаковых ветвей; без CAP используется та же конструкция 62° .
4	Дрокстал	SCALE: масштаб фиксированного шаблона; старая форма L D SCALE принимает, но игнорирует L, D .
10	Полый гексагональный столбик	L D CAVITY_DEG: верхняя и нижняя пирамидальные полости; угол должен быть положительным и меньше $\arctan(L/D)$.
12	Двухкомпонентный гексагональный агрегат	L D 2: ровно две одинаковые восьмигранные колонки в фиксированном взаимном положении.
999	Встроенный контрольный агрегат	SCALE: масштаб фиксированной регрессионной геометрии.

10.2. Внутренний формат частицы и экспорт геометрии

Первые три непустые строки имеют строгий смысл:

```
CONCAVE
AGGREGATE [FACETS_PER_PART]
BETA_DOMAIN_DEG GAMMA_DOMAIN_DEG

x0 y0 z0
x1 y1 z1
x2 y2 z2
```

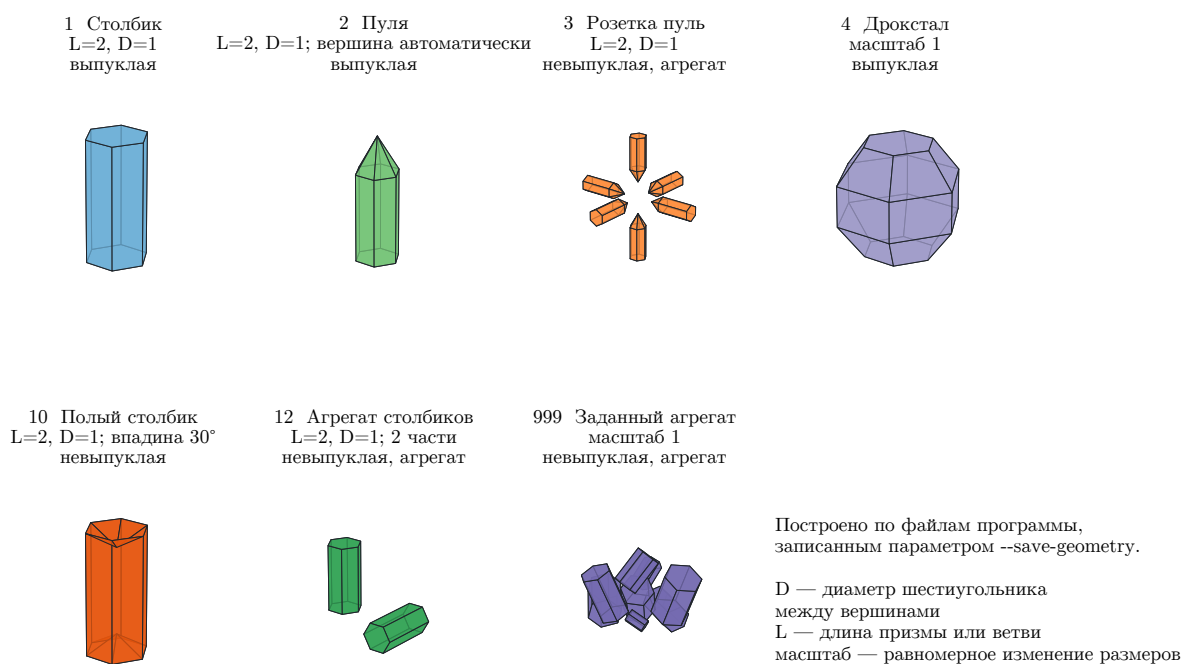


Рис. 5: Встроенные формы. Изображения построены по файлам из каталога `examples/particles`, записанным генераторами самой программы.

```
x0 y0 z0      # следующая грань
...
```

CONCAVE равен 0 или 1. AGGREGATE равен 0 для одной частицы либо записывается как 1 FACETS_PER_PART для компонентов с одинаковым числом граней. Третья строка задаёт ширину фундаментальной области по β и γ в градусах. Пустая строка разделяет грани; у каждой грани должно быть не меньше трёх конечных неколлинеарных вершин. Символ ## начинается комментарий. Порядок вершин определяет направление нормали, поэтому обход всех граней должен быть согласован.

Экспорт записывает координаты с 17 значащими цифрами и проверяет метаданные агрегата. Файл содержит уже масштабированную рабочую геометрию, поэтому подходит для точного повторения расчёта, визуализации обрезки пучков и проверки цикла «запись–чтение»:

```
gpu/bin/mbs_po_gpu_double_fast --method po \
  --particle 10 20 12 15 \
  --save-geometry concave.particle ...
gpu/bin/mbs_po_gpu_double_fast --method po \
  --particle-file concave.particle ...
```

Файл particle_for_check.dat создаётся автоматически в каталоге результата. Параметр --save-geometry удобнее, когда путь должен быть известен заранее.

10.3. Эквивалентный радиус и k_{eq}

Эквивалентный радиус по объёму:

$$r_{\text{eq}} = \left(\frac{3V}{4\pi} \right)^{1/3}.$$

Эквивалентный размерный параметр:

$$k_{\text{eq}} = \frac{2\pi r_{\text{eq}}}{\lambda}.$$

Если частица масштабируется параметром --k-eq X, то масштаб s выбирается из

$$s = \frac{k_{\text{eq,target}} \lambda}{2\pi r_{\text{eq,old}}}.$$

Таблица 10: Флаги размера.

Флаг	Смысл
--resize-dmax-um SIZE	Масштабировать геометрию из файла так, чтобы $D_{\text{max}} = \text{SIZE}$.
--k-eq X	Масштабировать так, чтобы $k_{\text{eq}} = X$. Требуется --wavelength-um.
--dmax-grid A B N	Логарифмический диапазон D_{max} .
--k-eq-grid A B N	Логарифмический диапазон k_{eq} .

Флаг	Смысл
--k-eq-list FILE	Точный список положительных k_{eq} из файла.

Часть III

Ориентации и угловые сетки

11. Ориентационное усреднение

11.1. Интеграл по ориентациям

Для случайно ориентированных частиц полная средняя по группе вращений имеет меру

$$\langle \mathbf{M} \rangle = \frac{1}{8\pi^2} \int_0^{2\pi} \int_0^\pi \int_0^{2\pi} \mathbf{M}(\alpha, \beta, \gamma) \sin \beta \, d\alpha \, d\beta \, d\gamma.$$

В основной двухугловой схеме MBS-fast интеграл по лабораторному углу α реализован через азимутальную сетку рассеяния. После этого явно усредняются β и γ :

$$\overline{\mathbf{M}}(\theta) = \frac{1}{W} \sum_{i=1}^{N_{\text{ori}}} w_i \mathbf{M}_i(\theta), \quad W = \sum_{i=1}^{N_{\text{ori}}} w_i.$$

В непрерывной форме редуцированная двухугловая часть равна

$$\overline{\mathbf{M}}(\theta) = \frac{1}{4\pi} \int_0^{2\pi} \int_0^\pi \mathbf{M}(\theta; \beta, \gamma) \sin \beta \, d\beta \, d\gamma.$$

Удобная переменная $\mu = \cos \beta$ дает

$$\sin \beta \, d\beta = -d\mu,$$

поэтому квадратура по μ естественным образом учитывает сферическую меру. Эта формула применима после усреднения по α ; сама по себе сетка $\beta \times \gamma$ не заменяет произвольную трёхугловую квадратуру.

11.2. Регулярная сетка по β, γ

Если ячейка по β имеет границы $\beta_{j-1/2}$ и $\beta_{j+1/2}$, её вес равен

$$w_\beta(j) = \cos \beta_{j-1/2} - \cos \beta_{j+1/2}.$$

Для равномерной сетки по γ

$$w_\gamma = \frac{\gamma_{\text{max}} - \gamma_{\text{min}}}{N_\gamma}.$$

Итоговый вес:

$$w_{j,l} = w_\beta(j)w_\gamma(l).$$

11.3. Флаг `--pole`

На полюсах $\beta = 0$ и $\beta = \pi$ все γ описывают одну и ту же физическую ориентацию:

$$\mathbf{R}_z(\gamma)\mathbf{R}_y(0) = \mathbf{R}_z(\gamma),$$

Поэтому `--pole` оставляет на полюсе одну точку по γ , но назначает ей вес всего интервала:

$$w_{\text{pole}} = w_{\beta} (\gamma_{\text{max}} - \gamma_{\text{min}}).$$

Эту специальную ветвь необходимо одинаково реализовывать на CPU и GPU. Расхождение полюсного веса или выбора базиса прежде всего проявляется в направлениях 0° и 180° .

11.4. Сетка по дифракционной оценке

Канонический параметр `--diffraction-sampling Q` строит сетки ориентаций и направлений рассеяния из одной оценки дифракционного углового масштаба:

$$\xi = 0.69 \frac{\lambda}{l_{\text{max}}}.$$

Здесь ξ выражена в радианах, а l_{max} — максимальная длина ребра среди всех граней уже масштабированной частицы:

$$l_{\text{max}} = \max_f \max_i \|\mathbf{r}_{f,i+1} - \mathbf{r}_{f,i}\|, \quad \mathbf{r}_{f,n_f} = \mathbf{r}_{f,0}.$$

Это не диаметр частицы и не максимальное расстояние между произвольными вершинами. Параметр Q задаёт число угловых интервалов на масштабе ξ :

$$\Delta_{\text{target}} = \frac{\xi}{Q}.$$

Следовательно, $Q = 1$ соответствует шагу ξ , $Q = 2$ — шагу $\xi/2$, а $Q = 4$ — шагу $\xi/4$. Чем больше Q , тем мельче сетка и дороже расчёт. Число Q не является общим числом ориентаций или числом точек на кольце.

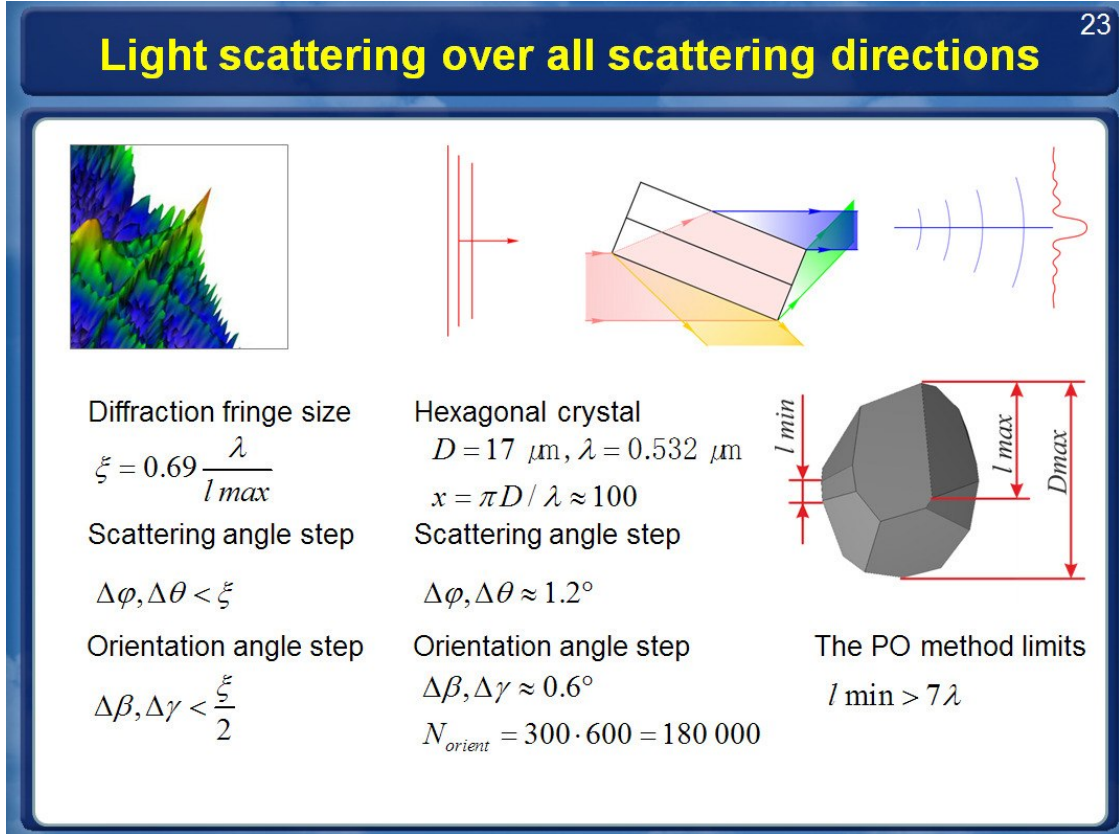


Рис. 6: Исходная оценка углового разрешения по дифракционной полосе. Показанное условие $\Delta\phi, \Delta\theta < \xi$ соответствует $Q_{\text{scat}} \geq 1$, а $\Delta\beta, \Delta\gamma < \xi/2$ — $Q_{\text{ori}} \geq 2$. В программе $l_{\max}$ вычисляется как максимальная длина ребра среди всех граней частицы, отдельно от полного диаметра $D_{\max}$.

Для ориентаций программа вычисляет

$$N_\beta = \max\left(3, \left\lceil \frac{\beta_{\max} - \beta_{\min}}{\xi/Q_{\text{ori}}} \right\rceil\right), \quad N_\gamma = \max\left(3, \left\lceil \frac{\gamma_{\max} - \gamma_{\min}}{\xi/Q_{\text{ori}}} \right\rceil\right).$$

Фундаментальные диапазоны β и γ берутся из симметрии частицы; --mirror-gamma дополнительно сокращает область по γ . Из-за округления фактические шаги могут быть немного меньше целевого ξ/Q .

Общий флаг назначает $Q_{\text{ori}} = Q_{\text{scat}} = Q$. Если требуется другая плотность только для ориентаций, применяется переопределение --orientation-diffraction-sampling q. Например,

```
--diffraction-sampling 2 \
--orientation-diffraction-sampling 4
```

задаёт $\xi/4$ для β, γ и сохраняет $\xi/2$ для θ, ϕ . Параметр --orientation-diffraction-sampling можно использовать без общего флага и совместить с явно заданным --scattering-grid.

Исторические параметры --diffraction-grid, --diffraction-limit-grid и --ring-points сохранены только для совместимости старых команд и показаны в --help-debug. Их нельзя смешивать с новой схемой: прежний алгоритм использовал другую зависимость числа ориентаций

от делителя и скрытое значение `ring-points=3`. Расчёты, выполненные до исправления определения $l_{\max}$, могли строить сетку по диаметру частицы; повторный запуск такой команды в новой версии правильно использует максимальное ребро и поэтому может выбрать другое число ориентаций.

11.5. Квазислучайные и случайные выборки

Квазислучайные режимы заменяют регулярную сетку набором точек низкой дискрепантности. Идея:

$$\overline{\mathbf{M}}_N = \frac{1}{N} \sum_{i=1}^N \mathbf{M}(T(\mathbf{u}_i)),$$

где $\mathbf{u}_i \in [0, 1)^d$ — Sobol/Hammersley/lattice point, а T переводит единичный куб в углы ориентации с правильной мерой.

Таблица 11: Ориентационные методы.

Флаг	Метод
<code>--fixed-orientation B G</code>	Одна ориентация β, γ ; удобна для отладки CPU/GPU.
<code>--orientation-file FILE</code>	Ориентации из файла.
<code>--euler-grid NB NG</code>	Явная регулярная сетка $\beta \times \gamma$.
<code>--diffraction-sampling Q</code>	Регулярная сетка ориентаций и автоматическая сетка рассеяния с шагом ξ/Q .
<code>--orientation-diffraction-sampling Q</code>	Только регулярная сетка ориентаций с шагом ξ/Q .
<code>--sobol N</code>	Квазислучайная последовательность Соболя.
<code>--sobol-seed N SEED</code>	Последовательность Соболя с заданным перемешиванием Оуэна.
<code>--so3-quaternion N</code>	Выборка Хаммерсли непосредственно на $SO(3)$ через кватернионы.
<code>--hammersley N</code>	Выборка Хаммерсли.
<code>--lattice N</code>	Ранговая решётка первого порядка.
<code>--monte-carlo N</code>	Псевдослучайные ориентации; ошибка обычно убывает как $N^{-1/2}$.

11.6. Адаптивный выбор числа ориентаций

При сравнении кандидатов по N_ϕ , глубине отражений и сетке Эйлера для двух приближений A и B ошибка одной строки по θ равна

$$e_{\text{row}} = \max \left(\frac{|M_{11}^A - M_{11}^B|}{\max(|M_{11}^A|, |M_{11}^B|, \varepsilon_0)}, \max_{(i,j) \neq (1,1)} \frac{|R_{ij}^A - R_{ij}^B|}{\max(1, |R_{ij}^A|, |R_{ij}^B|)} \right), \quad R_{ij} = \frac{M_{ij}}{M_{11}}.$$

Здесь $R_{ij} = M_{ij}/M_{11}$, а ε_0 защищает деление около нулей. Затем берётся максимум по строкам θ и добавляется относительное изменение интеграла $\int M_{11} d\Omega$. Для глубины отражений также проверяется выходная энергия. При подборе сетки θ значения в точках 1/3 и 2/3 каждого интервала сравниваются с интерполяцией, после чего отдельно проверяется интеграл M_{11} . При подборе числа точек Соболя проверяются входная энергия и все 16 элементов на контрольных углах.

По умолчанию для числа ориентаций, N_ϕ , глубины отражений и сетки Эйлера требуются два последовательных успешных уточнения; сетка θ уточняет интервалы напрямую. Параметр `--convergence-passes N` меняет число таких подтверждений. В совместных адаптивных режимах достижение верхнего предела считается ошибкой. Отдельный режим `--adaptive-orientations` выдаёт предупреждение и сохраняет лучший результат, но не подтверждает сходимость.

11.7. Файл настроек адаптивной сходимости

Параметр `--adaptive-config FILE` загружает все пределы рабочего расчёта из одного текстового файла. Полный шаблон с пояснением каждой строки находится в файле `configs/adaptive.example.conf`. Формат строгий: `key = value`; после символа `#` или `;` разрешен комментарий. Например:

```
reflections.min = 2
reflections.max = 30
reflections.tolerance = 0.01 # 1 percent
alpha.min_points = 12
alpha.max_points = 2400
alpha.tolerance = 0.01
orientations.min = 128
orientations.max = 262144
orientations.tolerance = 0.02
```

Группы `reflections`, `alpha`, `theta`, `orientations`, `beta` и `gamma` задают минимум, максимум и свою относительную точность. Группа `controller` задает число последовательных успешных уточнений, диапазон `pilot`-набора и максимальное число совместных проходов. Значение 0.01 означает относительную погрешность 1%, а не 0.01%.

Если индивидуальная точность отсутствует в файле, используется общий EPS из `--auto`, `--autofull` или другого выбранного режима. Индивидуальная точность из файла имеет приоритет над общим EPS. Явные флаги командной строки `--max-*`, `--max-reflections` и `--convergence-passes` имеют приоритет над файлом. Неизвестный или повторный ключ, неверный диапазон и неправильный тип значения останавливают запуск до расчета с указанием файла, номера строки и способа исправления.

```
cpu/bin/mbs_po_mpi --method po --particle 1 100 70 \
  --refractive-index 1.3116 0 --wavelength-um 0.532 \
  --autofull 0.02 \
  --adaptive-config configs/adaptive.example.conf \
  --backend cpu --threads 64 --output converged --close
```

11.8. Совместный адаптивный подбор параметров

Параметры не являются независимыми. Глубина n меняет дерево лучей; N_ϕ меняет эквивалентное усреднение по α ; сетка θ определяет, на каких углах контролируется результат; число ориентаций меняет каждую усреднённую строку. Поэтому общий порядок имеет вид

$$n \longrightarrow N_\phi \longrightarrow \{\theta_j\} \longrightarrow N_{\text{orient}}.$$

--auto EPS оставляет n фиксированным и подбирает N_ϕ , сетку θ и число точек Соболя. --autofull EPS дополнительно подбирает n . --diffraction-autofull EPS завершает поиск регулярной квадратурой Эйлера: N_β и N_γ уточняются по очереди, затем проверяются совместно. Цикл повторяется на увеличенном контрольном наборе, пока выбранные параметры не перестанут изменяться.

В совместном автоматическом режиме параметр --phi-points N фиксирует N_ϕ . Параметры --scattering-grid и --theta-grid-file фиксируют сетку θ ; остальные параметры продолжают подбираться. Чтобы искать эту размерность, соответствующий явный флаг надо убрать. Отдельные --adaptive-phi и --auto-phi по-прежнему несовместимы с --phi-points, потому что их единственная задача — подобрать N_ϕ .

--auto-theta-grid EPS проверяет две внутренние точки каждого интервала по полной матрице Мюллера и по интегралу M_{11} . В алгоритме нет заранее заданных углов особых областей: особенности конкретной формы должны обнаруживаться по рассчитанной кривой. Приближённая стоимость равна

$$\text{стоимость} \approx N_{\text{ori}} N_\theta N_\phi N_b.$$

Поэтому автоматический режим может быть заметно дороже одной заранее заданной сетки, но он даёт измеряемый критерий выбора параметров.

Каждая попытка записывается в файл <output>/<name>_convergence.tsv: параметр, номер совместного прохода, кандидат, максимальная ошибка, худший theta-угол, номер элемента и статус.

11.9. Почему симметрия столбика не делит N_ϕ на шесть

В коде используется соглашение

$$\mathbf{R} = R_z(\alpha)R_y(\beta)R_z(\gamma).$$

Цикл по азимуту рассеяния за счет вращательной ковариантности реализует усреднение по лабораторному углу α , поэтому в этом контексте $N_\phi = N_\alpha$. У шестигранного столбика есть осевая симметрия $R_z(2\pi/6)$ в системе частицы. Она уже сокращает область γ до

$$0 \leq \gamma < \frac{\pi}{3} = 60^\circ.$$

Но при $\beta \neq 0, \pi$ наклоненный столбик не инвариантен относительно поворота на 60° вокруг лабораторной оси падающего луча. Поэтому для α остается область $0 \leq \alpha < 2\pi$, и автоматическое деление N_ϕ на шесть было бы физически неверным. --alpha-points является понятным алиасом --phi-points, а --adaptive-alpha — алиасом --adaptive-phi; они не включают дополнительное приближение.

12. Угловая сетка рассеяния

12.1. Строки по θ и телесный угол

Равномерная сетка `--scattering-grid T0 T1 NPHI NTH` содержит $N_\theta + 1$ строк. Вес строки равен

$$\Delta\Omega_j = 2\pi [\cos\theta_{j,\text{left}} - \cos\theta_{j,\text{right}}].$$

Для полного диапазона $0^\circ \dots 180^\circ$:

$$\sum_j \Delta\Omega_j = 4\pi.$$

12.2. Азимутальная сетка

В полном расчёте используется N_ϕ азимутальных точек. В итоговом файле для случайных ориентаций обычно записан результат, уже усреднённый по азимуту, но внутри алгоритма ϕ остаётся отдельным измерением работы:

$$N_{\text{work}} \sim N_b N_\theta N_\phi N_{\text{ori}}.$$

12.3. Дифракционная оценка и широтная сетка по ϕ

В РО-режиме `--diffraction-sampling Q` использует для сетки рассеяния тот же масштаб $\xi_0 = 0.69\lambda/l_{\text{max}}$, что и для ориентаций. Целевой шаг равен

$$\Delta_{\text{scat}} = \frac{\xi_0}{Q_{\text{scat}}}.$$

Числа интервалов вычисляются как

$$N_\theta = \left\lceil \frac{\pi}{\Delta_{\text{scat}}} \right\rceil,$$

$$N_{\phi,\text{eq}} = \text{round_up}_6 \left[\max \left(12, \left\lceil \frac{2\pi}{\Delta_{\text{scat}}} \right\rceil \right) \right].$$

Результат содержит $N_\theta + 1$ строк от 0 до π . Значение $Q = 1$ использует дифракционный масштаб непосредственно, а $Q = 2$ уменьшает оба целевых угловых шага вдвое. Это априорная оценка разрешения, а не гарантия точности. Для итогового расчёта следует сравнить выбранное Q с удвоенным значением по всем требуемым элементам матрицы Мюллера.

Параметр `--scattering-diffraction-sampling Q` переопределяет только Q_{scat} . Например,

```
--diffraction-sampling 2 \
--scattering-diffraction-sampling 1
```

оставляет шаг ориентаций $\xi_0/2$, но задаёт шаг рассеяния ξ_0 .

Флаг `--latitude-phi-grid` сохраняет экваториальное разрешение, но задаёт отдельное число азимутальных точек для каждой строки:

$$N_\phi(\theta) = \min \left\{ N_{\phi,\text{eq}}, \text{round_up}_6 \left[\max \left(12, \left\lceil \frac{2\pi \sin \theta}{\Delta_{\text{scat}}} \right\rceil \right) \right] \right\}.$$

Так удаляются почти совпадающие азимутальные направления около полюсов без огрубления экватора. Число узлов округляется до кратного шести, а полюсная строка связывается с соседней рабочей группой для устойчивой обработки базиса. Накопление выполняется на полной прямоугольной выходной сетке $N_{\phi,eq}$, поэтому формат результата не меняется.

Автоматическая сетка рассеяния требует `--method po` и дифракционной сетки ориентаций. Она конфликтует с явными `--scattering-grid`, `--theta-grid-file`, `--auto-theta-grid` и `--phi-points`. Широтная сетка пока не поддерживается общим последовательным расчётом нескольких размеров. Каждый размер следует запускать отдельным процессом, чтобы l_{max} и сетка пересчитывались заново.

```
CUDA_VISIBLE_DEVICES=0,1,2,3 \
MBS_GPU_GROUPS=1 MBS_GPU_MULTI_MAX=4 \
gpu/bin/mbs_po_gpu_double_fast --method po \
  --particle 1 100 70 --refractive-index 1.3116 0 \
  --wavelength-um 0.532 --max-reflections 8 \
  --diffraction-sampling 2 \
  --latitude-phi-grid --backend cuda --threads 32 \
  --output results/column --close
```

12.4. Адаптивная сетка по θ

Передний дифракционный пик обычно требует более мелкого шага, чем боковые и задние углы. Режим `--auto-theta-grid EPS` не использует заранее назначенные зоны. Он сравнивает рассчитанные значения с интерполяцией в точках 1/3 и 2/3 каждого интервала, уточняет только не прошедшие интервалы и отдельно контролирует изменение интеграла M_{11} . Верхняя граница числа узлов задаётся параметром `--max-theta-points`.

Часть IV

Реализация и производительность

13. Реализация на CPU и AVX

13.1. Что параллелится на CPU

В CPU-версии OpenMP и MPI распределяют ориентации, строки сетки θ , пучки и группы размеров. К наиболее затратным операциям относятся вычисление фаз

$$\Phi = k(A_q x + B_q y), \quad \exp(i\Phi) = \cos \Phi + i \sin \Phi,$$

суммирование вкладов рёбер и накопление матриц Джонса и Мюллера.

13.2. Упаковка данных в AVX-512

Для AVX-512 в двойной точности:

$$\Phi = (\Phi_0, \Phi_1, \dots, \Phi_7),$$



Рис. 7: AVX-512 обрабатывает восемь значений двойной точности в одном векторном регистре; AVX2 обрабатывает четыре.

$$\mathbf{C} = \cos \Phi, \quad \mathbf{S} = \sin \Phi.$$

Одна векторная команда или векторизованный полиномиальный алгоритм обрабатывает сразу восемь направлений наблюдения либо восемь размеров. Для AVX2:

$$\Phi_{\text{AVX2}} = (\Phi_0, \Phi_1, \Phi_2, \Phi_3).$$

13.3. Почему важны флаги компиляции

Если компилятор не имеет права генерировать AVX-512, быстрый путь не включится. Для ERYC:

Таблица 12: Рекомендуемые CPU-флаги.

Платформа	ARCH_FLAGS	Комментарий
ЕРУС Zen2	<code>-march=znver2 -mtune=znver2</code>	AVX2/FMA.
ЕРУС Zen3	<code>-march=znver3 -mtune=znver3</code>	AVX2/FMA.
ЕРУС Zen4	<code>-march=znver4 -mtune=znver4</code>	AVX-512 при поддержке GCC/Clang.
ЕРУС Zen5	<code>-march=znver5 -mtune=znver5</code>	Лучший вариант для новых компиляторов.
Переносимый AVX2	<code>-O3 -mavx2 -mfma</code>	Для переносимого бинарного файла под AVX2.

Платформа	ARCH_FLAGS	Комментарий
Явный AVX-512	-O3 -mavx512f -mavx512dq -mavx512cd -mavx512bw -mavx512vl	Только если <code>lscpu</code> показывает эти возможности.

Флага `avx12` у GCC/Clang нет. Если имеется в виду EPYC Zen5 с AVX-512, используйте `-march=znver5`; если компилятор старый, используйте явные `-mavx512*` после проверки `lscpu`.

14. Реализация на GPU

14.1. Что ускоряет GPU

GPU ускоряет дифракционную часть, потому что она раскладывается на почти независимые элементы работы:

$$(b, \theta, \phi, o) \mapsto \mathbf{J}_{b,o}(\theta, \phi),$$

где b — номер пучка, а o — номер ориентации. До когерентного суммирования каждый вклад вычисляется независимо.

атомарный путь: потоки пучков складываются в общей ячейке Джонса

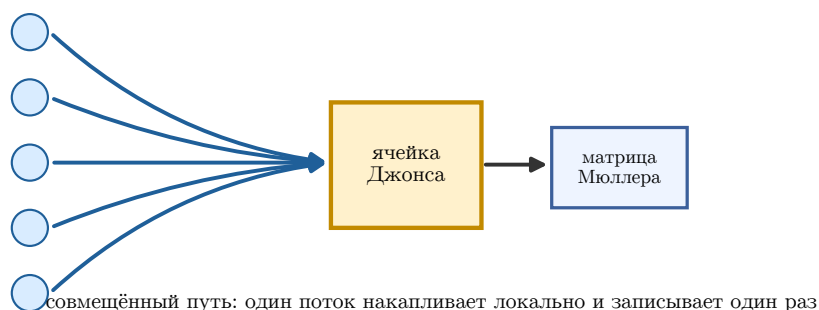


Рис. 8: Варианты CUDA-вычислений: атомарное накопление матриц Джонса, владение ячейкой без атомарных операций и совмещённое вычисление дифракции с преобразованием в матрицу Мюллера.

Таблица 13: Основные варианты ядер GPU.

Вариант	Работа одного потока	Область применения
Атомарное накопление	Вычисляет вклад пучка и добавляет его командой <code>atomicAdd</code> в матрицу Джонса.	Универсальный резервный и диагностический путь.
Владение ячейкой	Один поток владеет выходной ячейкой и последовательно проходит пучки.	Когда размещение данных позволяет избежать конкурирующих записей.
Совмещённое ядро	Локально суммирует матрицу Джонса и сразу преобразует её в матрицу Мюллера.	Основной путь при записи только полного результата.
Поэтапное накопление	Сначала строит матрицы отдельных ориентаций, затем редуцирует их.	Большие блоки ориентаций и высокая конкуренция атомарных операций.
Несколько k_{eq}	Обрабатывает близкие размеры из общего набора пучков.	Серии размеров с совместимыми геометрией и сеткой.

14.2. Атомарное накопление

Матрица Джонса имеет четыре комплексных элемента:

$$\mathbf{J} = \begin{pmatrix} J_{00} & J_{01} \\ J_{10} & J_{11} \end{pmatrix}, \quad J_{pq} = x_{pq} + iy_{pq}.$$

Атомарный вариант выполняет восемь сложений:

$$\text{atomicAdd}(x_{pq}), \quad \text{atomicAdd}(y_{pq}), \quad p, q \in \{0, 1\}.$$

Это корректно для когерентной суммы, но порядок сложения не детерминирован. Поэтому младшие разряды могут отличаться от CPU, особенно в одинарной точности и около вырожденных направлений.

14.3. Почему совмещённое ядро быстрее

Совмещённое ядро не записывает полный промежуточный массив матриц Джонса в глобальную память:

$$\sum_b \mathbf{J}_b \rightarrow \mathcal{M} \left(\sum_b \mathbf{J}_b \right) \rightarrow \text{глобальный массив Мюллера}.$$

Это уменьшает обмен с глобальной памятью и число атомарных операций, повышая вычислительную интенсивность. Эффект особенно заметен при больших N_b и N_ϕ .

Отдельное ускорение по азимуту даёт интерполяция `cuFFT`. Для автоматического выбора используется `--fft`, для явного и воспроизводимого сжатия `--fft-factor N`, а для измеряемого контроля на вложенной сетке `--fft-tolerance EPS`. Алгоритм, цена дополнительной проверки и смысл оценки ошибки разобраны в разделе 21.3.

14.4. Работа на нескольких GPU

При работе на нескольких GPU независимыми единицами распределения служат ориентации, группы строк θ или серии размеров. Идеальная оценка времени на G устройствах имеет вид

$$T_G \approx \frac{T_1}{G} + T_{\text{copy}} + T_{\text{reduce}}.$$

Линейному ускорению мешают обмены PCIe/NVLink, неодинаковая скорость устройств, размер блоков и финальное сложение матриц Мюллера.

Для переменной широтной сетки по ϕ `MBS_GPU_GROUPS=1` включает отдельный точный планировщик. Независимые группы строк θ назначаются видимым GPU динамически. Трассировка ориентаций и подготовленные пучки остаются общими в оперативной памяти; на каждой карте создаётся одна рабочая область CUDA. В пределах блока ориентаций упакованные пучки, веса и смещения загружаются один раз и повторно используются следующими группами строк.

Набор карт выбирается через `CUDA_VISIBLE_DEVICES`. Число исполнителей задаёт `MBS_GPU_MULTI=N`; значение 0 оставляет одну карту. Дополнительную верхнюю границу задаёт `MBS_GPU_MULTI_MAX=N`. Планировщик применяется только к прямой CUDA-дифракции без FFT-интерполяции и фильтрации пучков по зонам θ . Ускорение не обязано быть линейным: трассировка на CPU, упаковка, копирование по PCIe, редукция и дисбаланс групп θ остаются в полном времени.

Таблица 14: Переменные окружения для GPU.

Переменная	Смысл
<code>CUDA_VISIBLE_DEVICES=0,1</code>	Выбрать доступные GPU.
<code>MBS_GPU_FUSED_MUELLER=1</code>	Предпочитать совмещённое ядро дифракции и преобразования Мюллера.
<code>MBS_GPU_STAGE_MUELLER=1</code>	Включить поэтапное накопление по ориентациям.
<code>MBS_GPU_MULTI=0</code>	Отключить автоматическое распределение блоков ориентаций.
<code>MBS_GPU_MULTI_MAX=N</code>	Ограничить число исполнителей GPU.
<code>MBS_GPU_GROUPS=1</code>	Распределить группы θ широтной сетки по видимым GPU.
<code>MBS_GPU_MULTI_K_FULL=1</code>	Включить экспериментальное совмещённое вычисление нескольких k_{eq} .
<code>MBS_FFT_PHI_FACTOR=N</code>	Старый способ задания коэффициента FFT, только если не задан <code>--fft-factor</code> ; CLI имеет приоритет.
<code>MBS_SHARED_ORIENT_CHUNK=N</code>	Переопределить размер блока ориентаций.

Переменная	Смысл
MBS_BEAM_LOG=PATH	Записать первый обработанный набор РО-пучков; по умолчанию такой журнал не создаётся.

Часть V

Сборка, запуск и данные

15. Сборка и исполняемые файлы

Для CPU требуются GNU Make, компилятор GCC 9 или Clang 14 и новее с OpenMP, а также согласованная пара MPI-компилятора и запускающего процесса. Для GPU дополнительно нужны драйвер NVIDIA, набор CUDA с nvcc и библиотека cuFFT. После сборки следует сохранить вывод `--version`: в нём указаны версия исходного кода и включённые возможности CPU/CUDA.

15.1. CPU: MPI и OpenMP

```
make -C cpu -j
make test_release
cpu/bin/mbs_po_mpi --version
```

По умолчанию CPU-версия оптимизируется под текущий процессор. Для переносимого бинарного файла архитектуру задают явно:

```
make -C cpu clean
make -C cpu ARCH_FLAGS='-march=x86-64-v3 -mtune=generic' -j
```

15.2. GPU: точность и быстрые математические функции

```
make gpu_float -j
make gpu_float_fast -j
make gpu_double_fast -j
make gpu_double_fast_mpi -j
```

Таблица 15: Основные варианты сборки GPU.

Цель	Исполняемый файл	Назначение
gpu_float	gpu/bin/mbs_po_gpu_float	Одинарная точность без <code>--use_fast_math</code> .
gpu_float_fast	gpu/bin/mbs_po_gpu_float_fast	Наибольшая пропускная способность; требует калибровки ошибки.

Цель	Исполняемый файл	Назначение
gpu_double_fast	gpu/bin/mbs_po_gpu_double_fast	Двойная точность с CUDA --use_fast_math; основной рабочий вариант FP64.
gpu_double_fast_mpi	gpu/bin/mbs_po_gpu_mpi_double_fast	FP64 с MPI и CUDA для распределённого запуска.

Слово `fast` относится к параметру компилятора CUDA, а не к разрядности. Наиболее строгий контрольный GPU-бинарный файл двойной точности без быстрых математических функций собирается отдельно:

```
make -C gpu GPU_PRECISION=double GPU_FAST_MATH=0 \
  TARGET=bin/mbs_po_gpu_double -j
```

Новый класс частиц сначала проверяют в CPU FP64 и GPU FP64 без быстрых математических функций. Варианты `double_fast` и `float_fast` допускаются после измерения ошибки на характерных размерах, ориентациях и направлениях 0° и 180° .

16. Запуск и первичная диагностика

16.1. Рабочий запуск на GPU

```
CUDA_VISIBLE_DEVICES=0 \
gpu/bin/mbs_po_gpu_double_fast \
  --method po --particle 1 125.9 78.09 \
  --refractive-index 1.3116 0 --wavelength-um 0.532 \
  --max-reflections 12 --orientation-diffraction-sampling 2 \
  --scattering-grid 0 180 600 180 \
  --backend cuda --threads 16 \
  --output results/hex_gpu_double --close
```

16.2. Контроль CPU в том же GPU-бинарном файле

```
gpu/bin/mbs_po_gpu_double_fast \
  --method po --particle 1 125.9 78.09 \
  --refractive-index 1.3116 0 --wavelength-um 0.532 \
  --max-reflections 12 --orientation-diffraction-sampling 2 \
  --scattering-grid 0 180 600 180 \
  --backend cpu --threads 64 \
  --output results/hex_cpu_same_binary --close
```

Такое сравнение исключает различие интерфейса и настроек двух отдельных сборок. Для окончательного контроля его дополняют запуском `cpu/bin/mbs_po_mpi`.

16.3. Проверка направлений 0° и 180°

```

gpu/bin/mbs_po_gpu_double_fast \
  --method po --particle 1 125.9 78.09 \
  --refractive-index 1.3116 0 --wavelength-um 0.532 \
  --max-reflections 1 --orientation-diffraction-sampling 2 --pole \
  --scattering-grid 0 180 600 1 \
  --backend cuda --threads 16 \
  --output results/check_0_180 --close

```

17. Параметры командной строки

В таблицах ниже приведены канонические имена из `--help`. Сокращения `--po`, `-p`, `--pf`, `--ri`, `-w`, `-n`, `--oldauto`, `--diffraction-grid`, `--diffraction-limit-grid`, `--ring-points`, `--grid`, `--fixed`, `--random` и `-o` продолжают приниматься для старых сценариев, но в новых командах их использовать не следует. Полный перечень диагностических и экспериментальных параметров выводит `--help-debug`.

17.1. Метод, частица и оптические параметры

Таблица 16: Метод, геометрия и оптические параметры.

Параметр	Аргументы	Описание
<code>--method</code>	<code>po go</code>	Физическая или геометрическая оптика.
<code>--incoherent</code>	нет	Складывать матрицы Мюллера отдельных пучков без интерференционных членов.
<code>--particle</code>	<code>TYPE ...</code>	Создать встроенную частицу.
<code>--particle-file</code>	<code>FILE</code>	Прочитать геометрию из файла.
<code>--geometry</code>	<code>auto, convex</code> или <code>nonconvex</code>	Автоматически определить или явно задать класс геометрии.
<code>--save-geometry</code>	<code>FILE</code>	Сохранить фактически использованную геометрию.
<code>--resize-dmax-um</code>	<code>DMAX</code>	Масштабировать файловую частицу по $D_{\max}$, мкм.
<code>--k-eq</code>	<code>K</code>	Масштабировать по эквивалентному объёмному параметру.
<code>--refractive-index</code>	<code>REAL IMAG</code>	Комплексный показатель преломления.
<code>--wavelength-um</code>	<code>LAMBDA</code>	Длина волны, мкм.
<code>--max-reflections</code>	<code>N</code>	Максимальная глубина внутренних событий, от 0 до 30.

Параметр	Аргументы	Описание
--absorption	нет	Явно включить учёт поглощения; ненулевая мнимая часть также включает его.

17.2. Ориентации и адаптивная сходимость

Таблица 17: Основные способы задания ориентаций.

Параметр	Аргументы	Описание
--fixed-orientation	BETA GAMMA	Одна ориентация в градусах.
--orientation-file	FILE	Прочитать ориентации из файла.
--euler-grid	NBETA NGAMMA	Регулярная сетка $\beta \times \gamma$.
--euler-quadrature	NBETA NGAMMA	Квадратура Гаусса по $\cos \beta$ и периодическая сетка по γ .
--diffraction-sampling	Q	Задать шаг ξ/Q одновременно для ориентаций и рассеяния.
--orientation-diffraction-sampling	Q	Задать или переопределить шаг β, γ как ξ/Q .
--sobol	N	Квазислучайная последовательность Соболя.
--so3-quaternion	N	Выборка полной группы $SO(3)$ через кватернионы.
--adaptive-orientations	EPS	Подобрать только число ориентаций Соболя.
--adaptive-euler-grid	EPS	Раздельно и совместно уточнить N_β и N_γ .
--auto	EPS	Подобрать N_ϕ, θ и число ориентаций при фиксированном n .
--autofull	EPS	Дополнительно подобрать глубину n .
--diffraction-autofull	EPS	Совместный поиск с итоговой регулярной квадратурой Эйлера.
--adaptive-config	FILE	Прочитать отдельные пределы и допуски из файла.
--orientation-chunk	N	Ограничить число ориентаций в одном блоке памяти.
--pole	нет	На полюсе считать одно значение γ с полным весом.

17.3. Сетка рассеяния и вычислительная платформа

Таблица 18: Угловая сетка и выполнение расчёта.

Параметр	Аргументы	Описание
--scattering-grid	T0 T1 NPHI NTH	Равномерная сетка; записывается $NTH + 1$ строк.
--theta-grid-file	FILE	Неравномерная сетка θ из файла.
--auto-theta-grid	EPS	Адаптивно уточнить интервалы θ .
--scattering-diffraction-sampling	Q	Задать или переопределить шаги θ и экваториального ϕ как ξ/Q .
--latitude-phi-grid	нет	Использовать $N_\phi(\theta) \propto \sin \theta$.
--adaptive-phi	EPS	Подобрать N_ϕ , представляющее усреднение по α .
--phi-points	N	Явно зафиксировать N_ϕ .
--backend	auto cpu cuda	Выбрать вычислительную платформу.
--threads	N	Число потоков OpenMP на ведущем процессе.
--fft-factor	N	Рассчитать примерно в N раз меньше прямых точек ϕ и интерполировать их.
--fft-tolerance	EPS	Выполнить одно вложенное уточнение FFT-сетки.

17.4. Отсечения и серии размеров

Таблица 19: Управление стоимостью и сериями расчётов.

Параметр	Аргументы	Описание
--cutoff-profile	off, safe, balanced, fast	Согласованный набор порогов; не смешивать с отдельными пороговыми.
--beam-cutoff-importance	EPS	Отсечение выходного пучка по величине $ J ^2 A$.
--trace-cutoff-importance	EPS	Отсечение внутренней ветви по величине $ J ^2 A$.
--beam-area-ratio	R	Порог удаления малых фрагментов; профиль off отключает старое значение 100.
--trace-max-beams	N	Прервать ориентацию после N узлов дерева; 0 отключает предел.

Параметр	Аргументы	Описание
--dmax-grid	DMIN DMAX N	Логарифмическая серия по $D_{\max}$.
--k-eq-grid	KMIN KMAX N	Логарифмическая серия по k_{eq} .
--k-eq-list	FILE	Точный список значений k_{eq} .
--scan-jobs	N	Число дочерних процессов для серии размеров.
--gpu-devices	LIST	Список GPU для дочерних процессов серии.

17.5. Результаты и диагностика

Таблица 20: Вывод и служебные файлы.

Параметр	Аргументы	Описание
--output	PATH	Каталог и префикс результата.
--progress-interval	SECONDS	Период сообщений о ходе расчёта.
--save-betas	нет	Сохранять промежуточные матрицы для каждого кольца β .
--checkpoint	нет	Включить сохранение и продолжение поддерживаемых длинных расчётов.
--jones-output	нет	Записывать матрицы Джонса в поддерживаемых режимах.
--no-shadow-output	нет	Дополнительно записать результат без теневого пучка.
--close	нет	Завершить процесс после расчёта; сохранён для совместимости.
--help	нет	Показать поддерживаемые рабочие параметры.
--help-debug	нет	Показать также устаревшие, диагностические и экспериментальные параметры.

18. Выходные файлы и интегральные величины

Основной файл содержит угол θ , вес элемента телесного угла и элементы матрицы Мюллера. В файле они обозначены $M_{11}, \dots, M_{44}$, что соответствует $M_{00}, \dots, M_{33}$ в нулевой индексации кода.

Таблица 21: Основные выходные объекты.

Объект	Смысл
<code>theta</code>	Угол рассеяния в градусах.
<code>2pi*dcos</code>	Вес кольца телесного угла для данной строки.
<code>Mij</code>	Элементы Мюллера после суммирования пучков и ориентаций.
<code>*_noshadow</code>	Результат без теневого пучка при <code>--no-shadow-output</code> .
<code>*_betas/</code>	Промежуточные результаты по кольцам β при <code>--save-betas</code> .
<code>checkpoint</code>	Файлы продолжения поддерживаемых длинных расчётов.
<code>particle_for_check.dat</code>	Фактически использованная после масштабирования частица внутри каталога результата.
FILE из <code>--save-geometry</code>	Рабочая геометрия по явно выбранному пути.
<code><result>_integrals.tsv</code>	Машиночитаемая таблица сечений экстинкции, рассеяния и поглощения, эффективностей и оценки ошибки C_{sca} .

В журнал и TSV выводится по одному определению каждой физической величины для выбранного метода:

$$C_{\text{ext}}, \quad C_{\text{sca}}, \quad C_{\text{abs}}, \quad Q_{\text{ext}}, \quad Q_{\text{sca}}, \quad Q_{\text{abs}}, \quad Q_x = \frac{C_x}{A_{\text{proj}}}.$$

В физической оптике C_{ext} вычисляется по оптической теореме из амплитуды рассеяния вперёд. Для поглощающего материала $C_{\text{sca}} = \sum_j M_{11}(\theta_j) \Delta\Omega_j$ и $C_{\text{abs}} = C_{\text{ext}} - C_{\text{sca}}$. Для непоглощающего материала выполняется $C_{\text{sca}} = C_{\text{ext}}$ и $C_{\text{abs}} = 0$. В GO используется $C_{\text{ext}} = A_{\text{proj}}$; при поглощении C_{sca} равен полной энергии выходных пучков, а C_{abs} — их разности. В непоглощающей геометрической оптике снова принудительно выполняется физический баланс. Несогласованные старые определения и величины другого метода в отчёт не добавляются.

Оценка точности интеграла C_{sca} сравнивает полную сетку θ с вложенной сеткой из каждой второй строки:

$$\hat{\epsilon}_{\text{sca}} = \frac{|C_{\text{sca},h} - C_{\text{sca},2h}|}{\max(|C_{\text{sca},h}|, |C_{\text{sca},2h}|)}.$$

Для непоглощающего расчёта берётся максимум этой оценки и независимой невязки баланса: интеграл M_{11} сравнивается с оптической теоремой в физической оптике, а сумма выходных пучков — с площадью проекции в геометрической. При числе строк θ меньше пяти оценка недоступна. Значение больше 1% даёт статус `check_csca_integration`. Это апостериорная проверка сетки, а не строгая граница ошибки и не проверка сходимости по ориентациям.

TSV содержит поля `method`, `label`, `status`, шесть значений C/Q и `Csca_error_estimate_rel`; альтернативных старых определений в нём нет.

В начале и конце журнала записываются имя узла, модель CPU, число логических и физических ядер, фактическое число потоков OpenMP, общий и доступный объём ОЗУ, текущий и максимальный RSS процесса, видимые GPU и память активного устройства. В конце

также находятся отчёты FFT и отсечений. Время на занятом другим процессом GPU нельзя сравнивать с измерением на свободном устройстве.

Обычный запуск не создает диагностические файлы в текущем рабочем каталоге. Первый набор РО-пучков записывается только при явно заданном `MBS_BEAM_LOG=PATH`.

Часть VI

Точность, проверка и воспроизводимость

19. Обязательные проверки результата

1. Для сравнения CPU/GPU используйте один бинарный файл и меняйте только `--backend cpu|cuda`.
2. Для направлений 0° и 180° используйте `--pole` и двойную точность. Сетка `--scattering-grid` должна совпадать.
3. При измерении скорости фиксируйте платформу, точность и число GPU. Отдельно записывайте `MBS_GPU_FUSED_MUELLER` и `MBS_GPU_STAGE_MUELLER`.
4. После изменения коэффициентов Френеля, `RotateJones` или интеграла по рёбрам проверяйте не только M_{11} , но и M_{12} , M_{33} , M_{34} , M_{44} .
5. Для невыпуклой частицы или агрегата сравнивайте небольшие расчёты с `--trace-prefilter` и без него: результаты должны совпадать в пределах ошибки округления.

20. Оценка погрешности и выбор параметров

20.1. Нормировка интенсивности

В коде амплитуда пучка и дифракционный интеграл разделены: трассировка даёт $\mathbf{J}_{\text{beam}}$, а обработчик физической оптики домножает её на F_{edge} . Наблюдаемая интенсивность одного направления определяется совместно геометрией апертуры, фазовым интегралом, множителями Френеля, оптическим путём и общей нормировкой:

$$I(\theta, \phi) \propto \left\| \sum_{b=1}^{N_b} C_{\text{norm}} F_{\text{edge},b} \mathbf{R}_{\text{out},b} \mathbf{F}_{n,b} \right\|_F^2.$$

Здесь $\|\cdot\|_F$ — норма Фробениуса. При сравнении CPU и GPU надо сопоставлять весь вычислительный путь, а не только F_{edge} : части фазы хранятся в `J_phased`, `dirPhase` и накопленной матрице Джонса.

Таблица 22: Компоненты нормировки и где они могут проявляться.

Компонент	Практический смысл
C_{norm}	Общий множитель результата; не должен зависеть от выбранной платформы CPU или GPU.
F_{edge}	Дифракция апертуры; чувствительна к пределу в направлении вперёд.
$\mathbf{R}_{\text{out}}$	Поляризационный поворот; чувствителен к вырождению базиса рассеяния.
$\mathbf{F}_n$	Накопленная матрица Джонса и фаза оптического пути.
w_i	Вес ориентации; особенно важен при --pole.

20.2. Устойчивый предел интеграла по рёбрам

Проблемный случай интеграла по рёбрам возникает, когда

$$|A_q| \ll 1, \quad |B_q| \ll 1.$$

Тогда разность экспонент

$$\exp(i\Phi_{j+1}) - \exp(i\Phi_j)$$

вычитает почти равные числа. Устойчивая запись использует

$$\exp(iu) - \exp(iv) = \exp\left(i\frac{u+v}{2}\right) 2i \sin\left(\frac{u-v}{2}\right).$$

Если $\delta = (u - v)/2$ мал, то

$$\sin \delta \approx \delta - \frac{\delta^3}{6} + \frac{\delta^5}{120}.$$

Эта запись объясняет, почему направление точно вперёд должно обрабатываться отдельной ветвью: общая формула математически верна, но численно теряет значащие цифры. Для 0° и 180° такая потеря может выглядеть как различие CPU и GPU даже в двойной точности, если одна реализация использовала предельную формулу, а другая осталась в общей ветви.

20.3. Предел базиса Джонса на полюсах

Базис рассеяния зависит от

$$\mathbf{v}_t = \frac{\mathbf{v}_f \times \mathbf{s}}{\|\mathbf{v}_f \times \mathbf{s}\|}.$$

Если $\mathbf{s}$ параллелен $\mathbf{v}_f$, знаменатель равен нулю. При точном рассеянии вперёд или назад азимутальный базис не единственен. После соответствующего поворота базиса Стокса физическая матрица Мюллера должна быть инвариантна к этому выбору:

$$\mathbf{M}_\chi = \mathbf{R}_M(\chi) \mathbf{M}_0 \mathbf{R}_M(-\chi).$$

При усреднении случайных ориентаций многие азимутальные зависимости исчезают, но для одной ориентации они сохраняются. Поэтому отдельно проверяют совпадение M_{11} как интенсивности и совпадение поляризационных элементов в одной конвенции базиса.

20.4. Симметрия матрицы Мюллера вперёд и назад

Для случайных ориентаций и полной азимутальной симметрии рассеяние вперёд удовлетворяет ограничениям

$$M_{12} = M_{21} = M_{13} = M_{31} = 0,$$

а блок Q/U становится диагонально-симметричным в стандартном базисе. Ограничения для обратного направления зависят от конвенции параметров Стокса и реализации поворота на 180° . Помимо алгебраической симметрии надо проверять непрерывность угловых зависимостей:

$$\lim_{\theta \rightarrow 0^+} M_{ij}(\theta) = M_{ij}(0), \quad \lim_{\theta \rightarrow \pi^-} M_{ij}(\theta) = M_{ij}(\pi).$$

20.5. Ошибка ориентационного усреднения

Для метода Монте-Карло среднеквадратичная ошибка обычно имеет вид

$$\mathbb{E} \left[|\overline{M}_N - \overline{M}|^2 \right]^{1/2} \approx \frac{\sigma_M}{\sqrt{N}}.$$

Для последовательностей Соболя и Хаммерсли с малой дискрепантностью в гладких задачах часто наблюдается более быстрое убывание:

$$|\overline{M}_N - \overline{M}| \lesssim C \frac{(\log N)^d}{N},$$

Однако обрезка полигонов, каустики и предельные ветви делают интегранд негладким. Поэтому программа оценивает сходимость по реально вычисленным матрицам, а не по теоретической асимптотике.

20.6. Практическая формула выбора N_{ori}

Если известна грубая относительная вариация V требуемого элемента Мюллера, начальное число ориентаций для метода Монте-Карло можно оценить как

$$N_{\text{ori},0} = \left\lceil \left(\frac{V}{\varepsilon_{\text{target}}} \right)^2 \right\rceil$$

Для регулярной дифракционной сетки начальную оценку получают из углового масштаба:

$$N_\beta \approx \frac{\beta_{\max} - \beta_{\min}}{\Delta\theta_{\text{diff}}/\text{DIV}}, \quad N_\gamma \approx \frac{\gamma_{\max} - \gamma_{\min}}{\Delta\theta_{\text{diff}}/\text{DIV}}.$$

Итог:

$$N_{\text{ori}} \approx N_\beta N_\gamma,$$

с поправками на симметрию и --pole. Если симметрия частицы уменьшает область γ в s_γ раз, то

$$N_\gamma \leftarrow \left\lceil \frac{N_\gamma}{s_\gamma} \right\rceil.$$

20.7. Ошибка сетки по θ

Сетка θ должна разрешать самую узкую физическую особенность. Для большого размерного параметра x ширина переднего пика имеет порядок

$$\delta\theta \sim \frac{1}{x}.$$

При слишком крупном шаге интегральная интенсивность может быть приемлемой, но высота переднего пика и соседние строки будут неверными. Начальная оценка:

$$\Delta\theta_{\text{front}} \leq \frac{1}{5x} \frac{180}{\pi} \quad \text{или строже для контрольного расчёта.}$$

В задней полусфере допустим более крупный шаг, только если поляризационные элементы не имеют быстрых осцилляций. Режим `--auto-theta-grid` распределяет узлы по измеренной ошибке интерполяции.

20.8. Сходимость по глубине трассировки

Параметр `--max-reflections` контролирует число внутренних событий. Если R — характерный модуль коэффициента отражения Френеля, вклад поколения g приближённо убывает как

$$I_g \sim R^{2g}$$

для непоглощающего случая без фокусировок. С поглощением добавляется множитель

$$\exp(-2k \operatorname{Im}(m) L_g).$$

Сходимость по глубине проверяют по итоговой матрице Мюллера:

$$\varepsilon_n = \max_r \frac{\|\mathbf{M}_n(r) - \mathbf{M}_{n-1}(r)\|_\infty}{|M_{11,n}(r)| + \varepsilon_0}.$$

Если увеличение глубины слишком дорого, сначала калибруют отсечения пучков и анализируют число реально построенных ветвей.

21. Ускорения и контролируемые приближения

21.1. Отсечение малозначимых пучков

Выходной порог удаляет готовый пучок перед вычислением дифракции, а внутренний порог останавливает ветвь при построении дерева отражений и преломлений. Три безразмерных критерия имеют вид

$$j_b = \frac{\|\mathbf{J}_b\|_F^2}{J_{\text{ref}}^2} < \varepsilon_J, \quad a_b = \frac{A_b}{A_{\text{ref}}} < \varepsilon_A, \quad i_b = j_b a_b < \varepsilon_I.$$

Здесь $\mathbf{J}_b$ — накопленная матрица Джонса ветви, а A_b — площадь полигона пучка. Если включено несколько отдельных критериев, используется логическое «или»: достаточно одного условия. Поэтому счётчики причин в журнале могут перекрываться.

21.1.1. Единые профили

--cutoff-profile MODE нельзя смешивать с отдельными порогами отсечения. Он также несовместим с --beam-area-ratio: конфликт обнаруживается до начала расчета.

Таблица 23: Значения согласованных профилей отсечения.

Профиль	Выход i_b	Трасса i_b	Отношение площадей	Назначение
off	0	0	выключено	Эталон, сходимость и поиск ошибок.
safe	10^{-10}	10^{-12}	10^8	Первый рабочий кандидат после сравнения с off.
balanced	10^{-8}	10^{-10}	10^5	Массовые серии после калибровки данного класса частиц.
fast	10^{-6}	10^{-8}	10^3	Предварительные предварительные расчёты, не контрольные.
Старое значение	0	0	100	Совместимость, если профиль и отдельные пороги не указаны.

Профили используют совместную важность $|J|^2 A$, а не независимые пороги по амплитуде и площади. Поэтому большая слабая или малая интенсивная ветвь не удаляется только по одному множителю. Без явных параметров сохраняется старое упрощение с отношением площадей 100. Только --cutoff-profile off полностью отключает это приближение. Профили не задают --trace-max-beams.

21.1.2. Отдельные пороги

Таблица 24: Отдельные параметры отсечения.

Флаг	Диапазон	Действие
--beam-cutoff EPS	$[0, 1]$	Старый общий порог для j_b и a_b ; удаление по «или».
--beam-cutoff-jones EPS	$[0, 1]$	Удалить выходной пучок при $j_b < \text{EPS}$.
--beam-cutoff-area EPS	$[0, 1]$	Удалить выходной пучок при $a_b < \text{EPS}$.
--beam-cutoff-importance EPS	$[0, 1]$	Удалить выходной пучок при $i_b < \text{EPS}$.
--trace-cutoff EPS	$[0, 1]$	Старый общий порог внутренней ветви для j_b и a_b .
--trace-cutoff-jones EPS	$[0, 1]$	Остановить ветвь при $j_b < \text{EPS}$.

Флаг	Диапазон	Действие
--trace-cutoff-area EPS	$[0, 1]$	Остановить ветвь при $a_b < \text{EPS}$.
--trace-cutoff-importance EPS	$[0, 1]$	Остановить ветвь при $i_b < \text{EPS}$.
--beam-area-ratio R	$R \geq 1$	После невыпуклого разбиения оставить больший фрагмент при отношении площадей не меньше R .
--trace-max-beams N	$N \geq 0$	Жёсткий предел дерева одной ориентации; 0 отключает его. Достижение предела означает неполный результат.

Старый общий параметр можно сочетать с отдельным порогом Джонса или площади: отдельное значение имеет приоритет, а программа выдаёт предупреждение. Для согласованного профиля частичное переопределение запрещено.

Отчёты OUTPUT BEAM CUTOFF REPORT и TRACE CUTOFF REPORT показывают фактические пороги, число кандидатов, сохранённых и удалённых ветвей, причины и достижения пределов дерева. При калибровке для off и выбранного профиля фиксируют ориентации, n , θ и N_ϕ и сравнивают все 16 отношений M_{ij}/M_{11} . Доля удалённых пучков сама по себе не оценивает ошибку из-за когерентной интерференции.

21.2. Двойная и одинарная точность

Для одинарной точности машинное эpsilon имеет порядок

$$\varepsilon_{\text{float}} \approx 1.2 \cdot 10^{-7},$$

для двойной точности

$$\varepsilon_{\text{double}} \approx 2.2 \cdot 10^{-16}.$$

Если суммируется N вкладов похожей величины, ошибка порядка сложения грубо может расти как

$$\varepsilon_{\text{sum}} \sim N\varepsilon_{\text{mach}}$$

при неблагоприятном порядке сложения. Поэтому атомарное накопление FP32 может отличаться от CPU сильнее, чем FP64. Контроль около 0° и 180° следует выполнять сборкой FP64 без быстрых математических функций. Варианты double_fast и float_fast используют только после калибровки.

21.3. FFT-интерполяция по ϕ

Пусть N_ϕ — число азимутальных точек результата, а f — коэффициент сжатия. Первый проход CUDA непосредственно вычисляет примерно

$$N_{\phi, \text{direct}} = \max\left(16, \left\lfloor \frac{N_\phi}{f} \right\rfloor\right)$$

точек. Затем периодическое дополнение спектра Фурье нулями восстанавливает полную сетку. При $f = 1$, $N_\phi < 32$ или отсутствии фактического уменьшения сохраняется прямой путь без интерполяции.

--fft включает автоматический выбор f . Флаг --fft-factor N , $1 \leq N \leq 64$, явно задает степень сжатия и имеет приоритет над старой переменной окружения MBS_FFT_PHI_FACTOR. Восстановление предполагает ограниченный спектр:

$$f(\phi) = \sum_{m=-M}^M c_m e^{im\phi}.$$

Если существенны гармоники $|m| > M$, возникает наложение спектров или сглаживание. Большой коэффициент сжатия сам по себе не гарантирует точность.

--fft-tolerance EPS, $0 < \text{EPS} < 1$, добавляет вторую прямую сетку

$$N_{\phi,2} = \min(N_\phi, 2N_{\phi,\text{direct}}).$$

Код сравнивает значимые M_{11} и все 16 элементов Мюллера в масштабе M_{11} :

$$\hat{e} = \max \left(\max_{M_{11} \text{ significant}} \frac{|M_{11}^{(1)} - M_{11}^{(2)}|}{|M_{11}^{(2)}|}, \max_{i,j} \frac{|M_{ij}^{(1)} - M_{ij}^{(2)}|}{\max(|M_{11}^{(2)}|, 10^{-3} M_{11,\text{max}}^{(2)})} \right).$$

Если $\hat{e} > \text{EPS}$, сохраняется результат с удвоенным числом прямых точек ϕ . Это одно апостериорное уточнение, а не строгая верхняя граница ошибки и не итерационная проверка сходимости. Оно требует дополнительного дифракционного прохода и одновременно хранит грубый, уточнённый и выходной массивы, увеличивая время и максимальную память.

```
# Сжатие в 4 раза; одно уточнение при ошибке выше 1%
gpu/bin/mbs_po_gpu_float_fast ... --fft-factor 4 --fft-tolerance 0.01
```

Отчёт FFT INTERPOLATION REPORT содержит запрошенный и фактический коэффициенты, прямое и выходное N_ϕ , число проходов FFT, проверок и уточнений, а также наихудшую оценку. Значение $f = 1$ явно обозначает прямой расчёт. Для эталона параметры FFT убирают или задают --fft-factor 1; рабочее сжатие проверяют на характерных формах и размерах. Совмещённый путь нескольких k_{eq} не выполняет вложенную проверку: при заданном допуске диспетчер выбирает поддерживаемый прямой путь.

21.4. Повторное использование трассировки для нескольких размеров

Для ряда размеров геометрическое дерево пучков может совпадать, а фаза изменяется через kD :

$$\Phi_j(k, D) = kD \xi_j,$$

где ξ_j — безразмерная проекция вершины. Тогда можно повторно использовать нормированные вершины и коэффициенты рёбер:

$$x_j(D) = D \hat{x}_j, \quad y_j(D) = D \hat{y}_j.$$

Это ускоряет серию размеров, но требует контроля: если размер меняет пороги или автоматически выбранные сетки, повторное использование может перестать быть эквивалентным полной пересборке.

22. Диагностика и воспроизводимость

22.1. Поиск расхождения CPU/GPU

Если найдено отличие между CPU и GPU, его надо локализовать сверху вниз:

1. Использовать один бинарный файл и переключать только `--backend cpu|cuda`. Так исключаются различия интерфейса и настроек ведущего кода.
2. Зафиксировать двойную точность, `--scattering-grid`, `--fixed-orientation` и `--max-reflections`. На первом проходе отключить FFT и отсечения.
3. Проверить M_{11} отдельно от поляризационных элементов. Если отличается только поляризационный блок, вероятно ошибка поворота базиса.
4. Сравнить результат с `--pole` и без него. Отличие только в крайних направлениях указывает на полюсный вес или вырожденный базис.
5. Отключить совмещённое и поэтапное накопление, затем сравнить пути с атомарными операциями и без них. Различие младших разрядов связано с порядком редукции; отличие в процентах означает разные ветви алгоритма.
6. Если одна ориентация совпадает, а среднее нет, проверить веса, границы фундаментальной области и разбиение ориентаций на блоки.

22.2. Выбор параметров итогового расчёта

Параметры подбирают последовательно, изменяя на каждом шаге одну величину:

1. На небольшом размере подобрать `--max-reflections`, пороги отсечения и сетку рассеяния.
2. На среднем размере сравнить `--orientation-diffraction-sampling 2` и `--orientation-diffraction-sampling 4` при неизменной сетке рассеяния; при заметном отличии продолжить уточнение или использовать адаптивный режим.
3. На нескольких характерных размерах сравнить одинарную и двойную точность, а также сборки с быстрыми математическими функциями и без них.
4. Распределение по нескольким GPU, FFT и повторное использование трассировки включать только после проверки эквивалентного прямого расчёта.
5. Вместе с результатом сохранить полную команду, ревизию Git, версии компилятора, CUDA и драйвера, модель GPU и переменные окружения.

22.3. Проверка геометрии из файла

Точный синтаксис собственного формата приведён в разделе «Внутренний формат частицы и экспорт геометрии». После чтения любая частица представлена набором вершин и упорядоченных граней:

$$\mathcal{P} = \left(\{\mathbf{v}_i\}_{i=1}^{N_v}, \{\mathcal{F}_j\}_{j=1}^{N_f} \right).$$

Грань $\mathcal{F}_j$ является упорядоченным списком индексов вершин:

$$\mathcal{F}_j = (i_{j,1}, i_{j,2}, \dots, i_{j,m_j}).$$

Нормаль грани вычисляется из обхода вершин. Для треугольной части:

$$\mathbf{n}_j = \frac{(\mathbf{v}_{i_2} - \mathbf{v}_{i_1}) \times (\mathbf{v}_{i_3} - \mathbf{v}_{i_1})}{\|(\mathbf{v}_{i_2} - \mathbf{v}_{i_1}) \times (\mathbf{v}_{i_3} - \mathbf{v}_{i_1})\|}.$$

Обратный обход меняет знак нормали. Это критично: ветвление по коэффициентам Френеля, определение внутренней стороны и обрезка пучков используют ориентацию нормали. У невыпуклой частицы грани не должны самопересекаться, а соседние полигоны должны иметь согласованную топологию.

Таблица 25: Проверки геометрии из файла перед расчётом.

Проверка	Зачем нужна
Нет нулевых рёбер	Иначе нормаль и коэффициенты рёбер становятся неопределёнными.
Грань плоская	Апертурный интеграл предполагает плоский полигон.
Нет самопересечения грани	Обрезка и формула площади требуют простого полигона.
Согласованный обход	Нужен правильный знак нормали.
Нет дублированных почти совпадающих граней	Иначе трассировка даёт повторные пересечения и неустойчивую обрезку.
Разумный масштаб	Слишком малые или большие координаты ухудшают обусловленность.

22.4. Площадь, объём и центр

Площадь полигона в локальной плоскости:

$$A_j = \frac{1}{2} \left| \sum_{l=1}^{m_j} (x_l y_{l+1} - x_{l+1} y_l) \right|.$$

Центр грани:

$$\mathbf{c}_j = \frac{1}{m_j} \sum_{l=1}^{m_j} \mathbf{v}_{i_{j,l}}.$$

Объём замкнутого многогранника удобно выражается через тетраэдры с началом координат:

$$V = \frac{1}{6} \sum_{\text{triangles}} \mathbf{a} \cdot (\mathbf{b} \times \mathbf{c}).$$

Знак объёма зависит от ориентации обхода. Отрицательное или близкое к нулю значение для заведомо объёмной частицы почти всегда указывает на ошибку геометрии.

22.5. Проверка чтения выходной строки

Типовая выходная строка содержит угол, вес и 16 элементов матрицы Мюллера:

$$\theta_j, \quad \Delta\Omega_j, \quad M_{11}, M_{12}, \dots, M_{44}.$$

В терминах кода:

$$M_{11} \equiv M_{00}, \quad M_{44} \equiv M_{33}.$$

Если результат усреднён по ориентациям, каждый элемент уже содержит сумму

$$M_{ab}^{\text{out}}(\theta_j) = \frac{1}{W} \sum_i w_i M_{ab}^{(i)}(\theta_j).$$

Если затем нужна интегральная характеристика по углам, используется вес $2\pi \cdot d\cos$:

$$\langle M_{ab} \rangle_\Omega = \sum_j M_{ab}^{\text{out}}(\theta_j) \Delta\Omega_j.$$

Не надо повторно умножать на 2π , если колонка уже называется $2\pi \cdot d\cos$.

22.6. Матрица контрольных тестов

Таблица 26: Минимальная матрица проверок после изменения физического кода.

Тест	Команда/режим	Что должен поймать
Одна ориентация	<code>--fixed-orientation</code> В G	Ошибки поворота Джонса, коэффициентов Френеля и интеграла по рёбрам.
Только крайние углы	<code>--scattering-grid 0 180 N 1</code> <code>--pole</code>	Полусный вес и вырожденный базис.
CPU и GPU одного бинарного файла	<code>--backend cpu cuda</code>	Различия платформ при одинаковом ведущем коде.
Двойная и одинарная точность	Бинарные файлы <code>double</code> и <code>float</code>	Потери точности и влияние быстрых функций.
Некогерентная сумма	<code>--incoherent</code>	Ошибки порядка суммирования Джонса и Мюллера.
Невыпуклая обрезка	Предварительный отбор включён и выключен	Ошибки отбрасывания по проекциям AABV.
Сходимость ориентаций	<code>--orientation-diffraction</code> <code>-sampling 1, 2, 4</code>	Недостаточное число ориентаций.
Сходимость глубины	<code>--max-reflections N</code> и <code>N+1</code>	Недостаточная глубина трассировки.

22.7. Типичные причины больших расхождений

Таблица 27: Диагностика расхождений.

Симптом	Вероятная причина	Что проверить
Отличается только 0°	Полюсная ветвь	--pole, границы ячейки θ , предельный базис.
Отличается только 180°	Обратный базис или знак Френеля	--legacy-sign, BackwardScattering, поворот Мюллера.
M_{11} совпадает, M_{33} и M_{34} нет	Соглашение о поляризованном базисе	RotateJones и экспериментальная ветвь Карчевского.
Одинарная точность заметно отличается	Компенсация больших вкладов или быстрые функции	Околопередние строки и порядок атомарной редукции.
GPU отличается только на большой сетке	Порядок редукции или разбиение на блоки	Совмещённый и поэтапный пути, размер блока.
Невыпуклый результат нестабилен	Ошибка кандидатов на обрезку	Предварительный отбор, нормали граней, самопересечения.
Автоматический и ручной режимы различаются	Разные сетки углов и ориентаций	Сохранить выбранные сетки и повторить их явно.

22.8. Контроль интегрального баланса

Для непоглощающих частиц необходимо контролировать интегральные величины, хотя приближения физической и геометрической оптики не обязаны выполнять точный баланс на любой конечной сетке. Если $S_{11}(\theta)$ соответствует M_{11} , численная величина рассеяния равна

$$C_{\text{sca,num}} = \sum_j S_{11}(\theta_j) \Delta\Omega_j.$$

При уточнении сеток θ и ориентаций эта величина должна стабилизироваться. Скачки при небольшом изменении N_θ указывают на неразрешённый передний пик.

22.9. Минимальные данные для воспроизводимого отчета

Вместе с результатом сохраняют:

Таблица 28: Данные для воспроизводимости.

Данные	Почему нужны
Ревизия Git	Физические формулы и реализация CUDA могли измениться.

Данные	Почему нужны
Полная команда	Значения по умолчанию могут различаться между версиями.
Компилятор и ARCH_FLAGS	Векторизация и слияние операций зависят от сборки.
Версии CUDA и драйвера	Они определяют реализацию библиотек и математических функций.
Модель GPU	Скорость FP32, FP64 и атомарных операций различается.
Переменные окружения	Через них выбираются совмещённый, поэтапный и многокарточный режимы.
Хеш файла частицы	Геометрия должна совпадать побайтно.

Часть VII

Приложения

А. Связь с файлами кода

Таблица 29: Где искать описанные объекты в коде.

Файл/область	Что там находится
src/main.cpp	Запуск расчёта, адаптивные режимы и справка.
src/Options.cpp	Канонические имена параметров, алиасы и группы справки.
src/RunConfig.cpp	Проверка совместимости параметров и построение конфигурации запуска.
src/Splitting.cpp	Коэффициенты Френеля, направления отражённой и прошедшей ветвей, оптический путь.
src/Beam.*	Геометрия пучка, матрица Джонса и состояние полигона.
src/scattering	Пересечения луча с гранью и обрезка для выпуклых и невыпуклых частиц.
src/handler/Handler.cpp	Скалярные и векторизованные интегралы дифракции по рёбрам.
src/handler/HandlerPO.cpp	Поворот Джонса, дифракция, накопление Мюллера и пути AVX.
src/math/Mueller.cpp	Преобразование Джонса в Мюллера и повороты матрицы Мюллера.
src/cuda и файлы *.cu	Ядра CUDA, редукции и совмещённые вычислительные пути.

В. Контрольные формулы для отчётов

В.1. Относительная ошибка элемента Мюллера

Для сравнения двух расчетов A и B удобно печатать относительную ошибку по каждому элементу:

$$\delta_{ij}(\theta) = \frac{M_{ij}^A(\theta) - M_{ij}^B(\theta)}{|M_{11}^A(\theta)| + \varepsilon_0}.$$

Нормировка на M_{11} полезна, потому что слабые поляризационные элементы могут проходить через ноль. Для абсолютного контроля слабых элементов дополнительно используют

$$\Delta_{ij}(\theta) = M_{ij}^A(\theta) - M_{ij}^B(\theta).$$

В отчетах лучше указывать обе величины: относительная ошибка показывает физический масштаб, абсолютная ошибка показывает численный шум около нулей.

В.2. Интегральная ошибка по углам

Если сравниваются полные угловые зависимости, используется взвешенная среднеквадратичная ошибка:

$$E_{ij}^2 = \frac{\sum_j (M_{ij}^A(\theta_j) - M_{ij}^B(\theta_j))^2 \Delta\Omega_j}{\sum_j (|M_{11}^A(\theta_j)| + \varepsilon_0)^2 \Delta\Omega_j}.$$

Такая метрика не определяется одной точкой, но направления 0° и 180° всё равно проверяют отдельно: для них используются специальные численные пределы.

В.3. Сравнение времени

Время расчёта раскладывают на компоненты:

$$T_{\text{total}} = T_{\text{trace}} + T_{\text{diffraction}} + T_{\text{reduction}} + T_{\text{io}}.$$

GPU ускоряет прежде всего $T_{\text{diffraction}}$ и часть $T_{\text{reduction}}$. Для сложной невыпуклой частицы может доминировать T_{trace} , поэтому общее ускорение будет ниже. При сравнении CPU и GPU фиксируют

$$N_{\text{ori}}, \quad N_\theta, \quad N_\phi, \quad N_b, \quad n, \quad \text{пороги отсечения.}$$

В.4. Оценка памяти GPU

Грубая оценка памяти для массива матриц Джонса:

$$B_J \approx N_\theta N_\phi N_{\text{ori,chunk}} \cdot 4 \cdot 2 \cdot s_{\text{real}},$$

где 4 — число комплексных элементов Джонса, 2 — действительная и мнимая части, а $s_{\text{real}} = 4$ для одинарной и 8 для двойной точности. Для массива матриц Мюллера

$$B_M \approx N_\theta N_\phi N_{\text{ori,chunk}} \cdot 16 \cdot s_{\text{real}}.$$

Совмещённые ядра уменьшают потребность в полном B_J : сумма Джонса остаётся в регистрах или локальной памяти до преобразования в матрицу Мюллера.

С. Контрольный список перед публикацией результатов

1. Указать ветку и ревизию Git.
2. Указать исполняемый файл CPU/GPU, точность и использование быстрых математических функций.
3. Указать источник частицы: параметры встроенного генератора или хеш файла геометрии.
4. Указать λ , $m = n + i\kappa$, размер и `--max-reflections`.
5. Указать способ усреднения и фактическое число ориентаций.
6. Указать сетки θ , ϕ и использование `--pole`.
7. Указать пороги отсечения и параметры обрезки невыпуклых частиц.
8. Указать переменные окружения GPU.
9. Приложить сравнение хотя бы двух разрешений угловой и ориентационной сеток.
10. Для быстрого GPU-режима приложить контроль двойной точности на подвыборке.

С.1. Минимальная таблица в статье или отчете

Таблица 30: Минимальные поля таблицы воспроизводимости.

Поле	Пример
Код	MBS-fast, commit abcdef0.
Платформа	gpu/bin/mbs_po_gpu_double_fast.
Частица	<code>--particle 1 125.9 78.09</code> или <code>--particle-file particle.dat</code> .
Оптика	<code>--refractive-index 1.3116 0 --wavelength-um 0.532</code> <code>--max-reflections 12</code> .
Ориентации	<code>--orientation-diffraction-sampling 2 --pole</code> .
Сетка рассеяния	<code>--scattering-grid 0 180 600 180</code> .
Оборудование	Модели CPU и GPU, версии драйвера и CUDA.
Проверка	Максимальная ошибка CPU/GPU, ошибка при уточнении ориентаций и глубины.

С.2. Что не надо смешивать в одном сравнении

Нельзя одновременно менять вычислительную платформу, точность, сетку ориентаций и сетку θ , а затем приписывать всё отличие GPU. В корректной серии каждый переход меняет только один фактор:

$$A_0 \rightarrow A_1 \rightarrow A_2 \rightarrow \dots,$$

где каждая стрелка меняет только один параметр. Например:

CPU, двойная точность $\rightarrow$ GPU, двойная точность
 $\rightarrow$ GPU, одинарная точность
 $\rightarrow$ GPU, одинарная точность и быстрые функции.